

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 13 日

浙江大学实验报告

课程名称：____计算机组成与设计____ 实验类型：____综合____

实验项目名称：____流水线处理器集成____

学生姓名：____张序鸿____ 专业：____混合班____ 学号：3230103265

指导老师：____杨莹春、潘之杰____ 助教：____马致远____

实验地点：____东 4-509____ 实验日期：2024 年 12 月 11 日

一、实验目的和要求

1. 理解流水线 CPU 的基本原理和组织结构
2. 掌握五级流水线的工作过程和设计方法
3. 理解流水线 CPU 停机的原理
4. 设计流水线测试程序

二、实验内容和原理

内容：

任务一：集成设计流水线 CPU，在 Exp04 的基础上完成

- 利用五级流水线各级封装模块集成 CPU
- 替换 Exp04 的单周期 CPU 为本实验集成的五级流水线 CPU

任务二：设计流水线测试方案并完成测试

原理：

2.1 五级流水线 – 具体操作

五级流水线主要分为 IF（取指）、ID（译码）、EXE（执行）、MEM（内存读写）、WB（寄存器读写）：

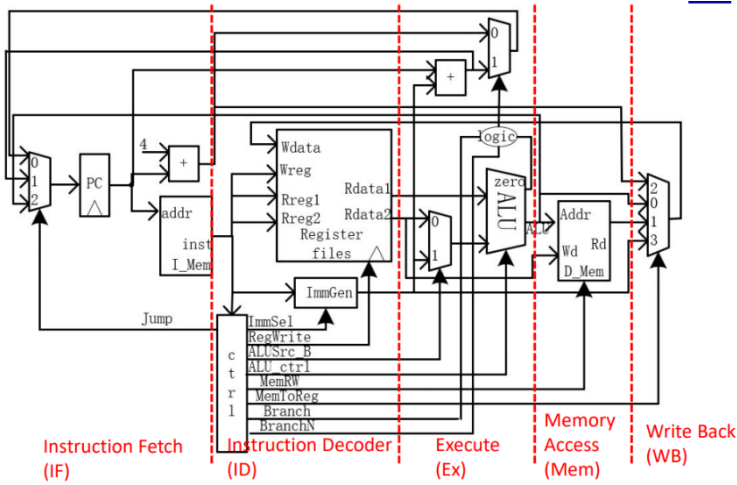
指令 \ 流水段	ALU	LOAD/STORE	BRANCH
IF	取指令	取指令	取指令
ID	译码，读寄存器文件	译码，读寄存器文件	译码，读寄存器文件
EXE	执行	计算访存有效地址	计算转移目标地址，设置条件码
MEM	(空操作)	对存储器读或写操作	若条件成立，将转移地址送 PC
WB	结果写入寄存器文件	读出数据写入寄存器文件	(空操作)

连续指令对应五级流水线时空图：

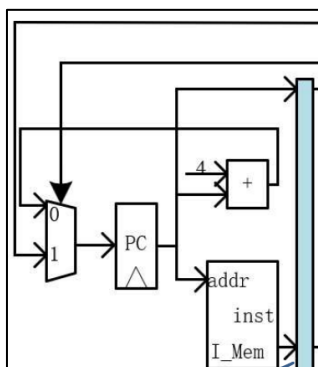
指令序列	流水时钟数								
	1	2	3	4	5	6	7	8	9
指令 i	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	EX	MEM	WB			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

2.2 五级流水线 Datapath

Pipeline 流水线构建的 Datapath 区别单周期 CPU 中的 Datapath 在于增加了若干寄存器（如 IF_reg_ID、ID_reg_EX、EX_reg_MEM、Mem_reg_WB），它们负责储存 5 个流水线上级的数据，从而实现每个时钟都能读取一条指令，并将上一级的数据传递到下一级。



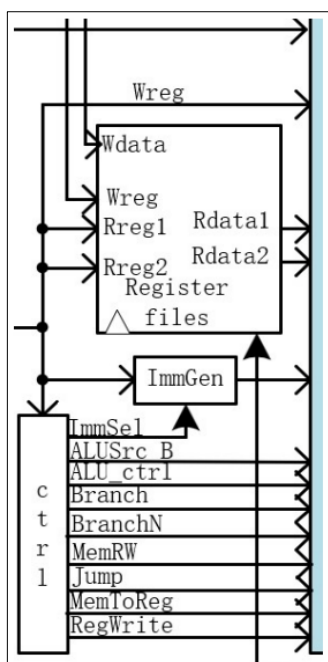
(1) Instruction Fetch (IF):



IF: 取指阶段涉及程序计数器 (PC) 和指令存储器 (I_Mem), 程序计数器输出作为地址从指令存储器中读取指令。

IF_reg_ID: 暂存指令、PC, 以待下一级使用。

(2) Instruction Decoder (ID):

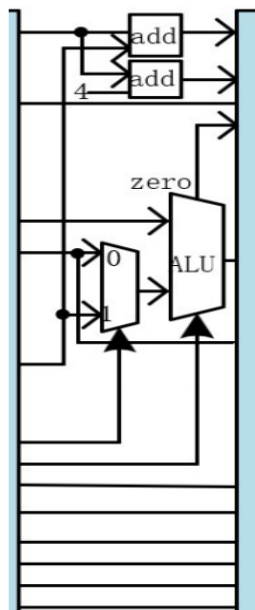


ID: 译码阶段涉及寄存器堆 (RegisterFiles) 和译码器 (SCPU_ctrl)、立即数生成单元 (ImmGen)。

从寄存器堆可以读取操作数后, 译码器对指令进行解析产生各种各种控制信号, 立即数生成单元根据控制信号和输入指令生成各种类型的立即数。

ID_reg_Ex: 暂存 PC 值, 寄存器读取的数据 (rs1, rs2), 立即数和控制信号以待下一级使用

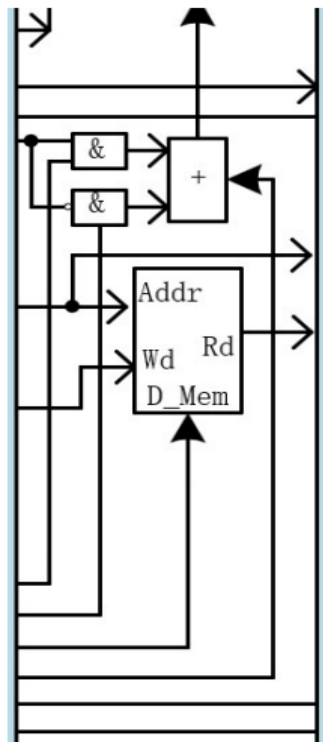
(3) Execute (Ex):



Ex : 执行阶段涉及运算单元 (ALU) 它获取操作数并完成指定的算数运算或逻辑运算。

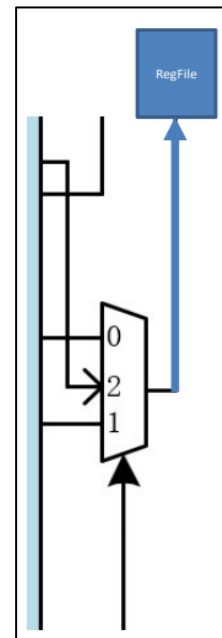
Ex_reg_Mem: 暂存运算结果和控制信号, 以待下一级使用。

(4) Memory Access (Mem):



Mem：存储器访问阶段涉及数据存储器（D_Mem），Load\Store 指令对数据存储器进行读或写。

Mem_reg_WB: 暂存存储器结果 和控制信号,
以待下一级使用



(5) Write Back (WB):

WB: 写回阶段涉及寄存器堆 (RegisterFiles), 将 ALU 的运算结果、存储器输出结果、PC+4 写回到寄存器堆。

此处不存在寄存器，写回阶段结束，一次完整的五级流水线操作完成！

2.3 五级流水线 SCPU_ctrl

流水线的控制信号同单周期 CPU 一样，来自于对指令的译码操作输出。

不同点在于流水控制信号会根据每阶段的不同功能部件选择性控制输出，本阶段用不到的信号会暂存于**寄存器**，并逐级传递下去。

(1) IF 控制:

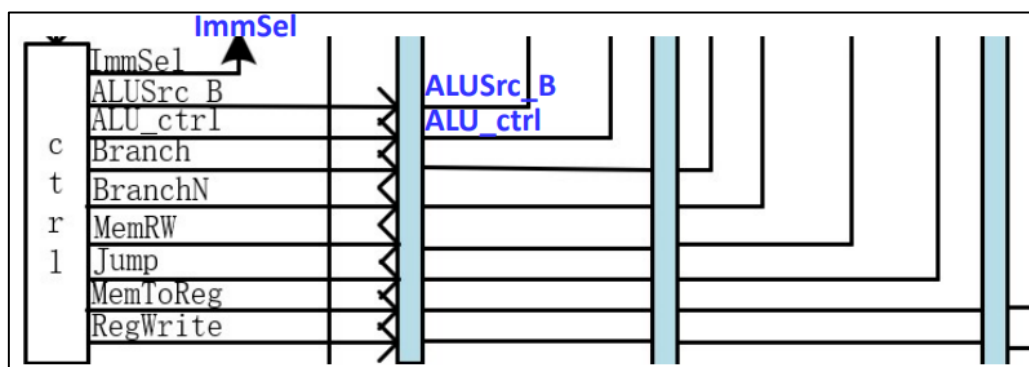
取指阶段，读指令存储器和写 PC 值永远有效，无需控制信号。

(2) ID 控制:

译码阶段，立即数生成单元需要根据指令类型产生对应输出，由控制信号 ImmSel 信号输出；其他信号暂存寄存器。

(3) Ex 控制:

执行阶段，ALU 的操作和第二个操作数 Src_B 需要选择，由控制信号 ALU_ctrl、ALUSrc_B 决定；其他信号暂存寄存器。

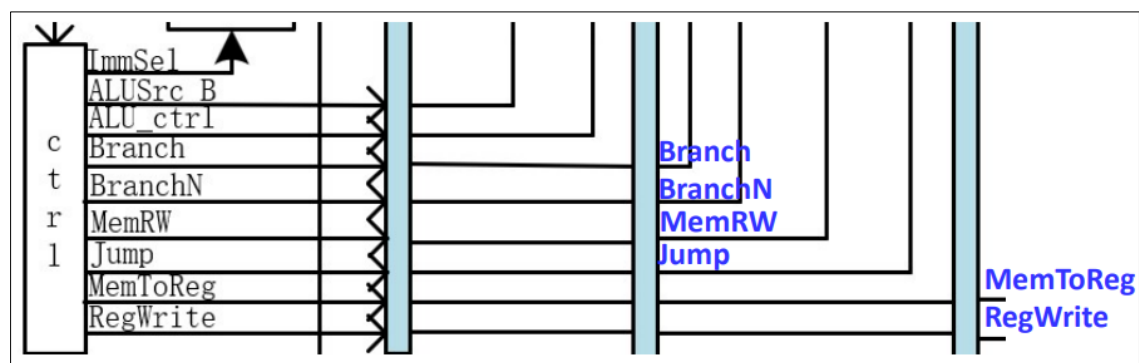


(4) Mem 控制:

存储器访问阶段，需要读写存储器以及根据分支 跳转指令判定选择 PC 转移值，MemRW、Branch、BranchN、Jump 输出；其他信号暂存寄存器。

(5) WB 控制:

写回阶段，ALU 运算结果、存储器输出等需要选择写回寄存器堆，同时寄存器堆的写使能需要设置；MemToReg、RegWrite 信号输出。

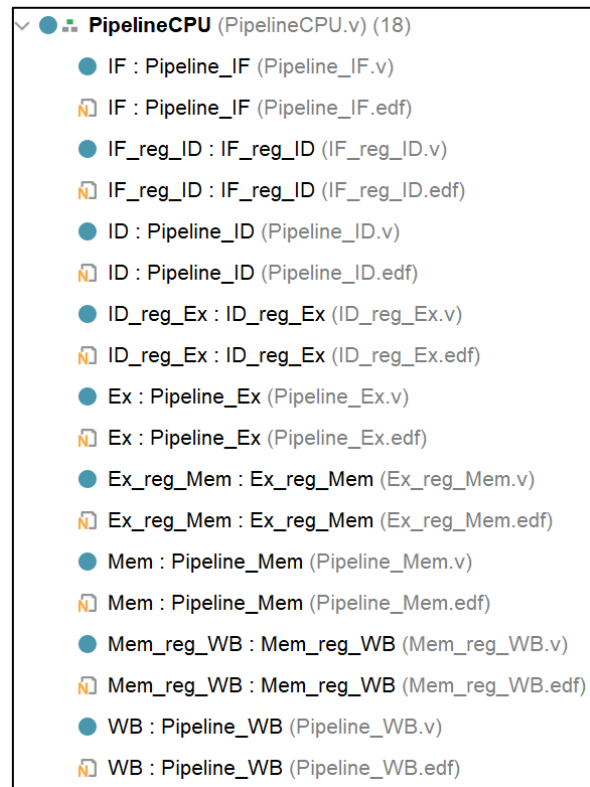


三、实验过程和数据记录

3.1 集成设计流水线 CPU:

创建 Pipeline_CPU.v 文件，将提供的流水线各阶段文件（.edf 文件）集成。

首先展示一下 Pipeline_CPU 的工程结构：



• PipelineCPU 接口设计:

```
module PipelineCPU(  
    input clk,           //时钟  
    input rst,           //复位  
    input[31:0] Data_in, //存储器数据输入  
    input[31:0] inst_IF,  //取指阶段指令  
    output [31:0] PC_out_IF, //取指阶段 PC 输出  
    output [31:0] PC_out_ID, //译码阶段 PC 输出  
    output [31:0] inst_ID, //译码阶段指令  
    output [31:0] PC_out_IDEX, //执行阶段 PC 输出  
    output [31:0] inst_EX, // 执行阶段指令  
    output MemRW_EX, //执行阶段存储器读写  
    output MemRW_Mem, //访存阶段存储器读写  
    output [31:0] Addr_out, //地址输出  
    output [31:0] Data_out, //CPU 数据输出  
    output [31:0] Data_out_WB, //写回数据输出  
    output [1023:0] all_regs_data  
);
```

- 流水线 CPU 执行过程中的传递信号:

```

•   wire [31:0] Rd_addr_out_ID , Rs1_out_ID , Rs2_out_ID,
    Imm_out_ID;
•   wire [3:0] ALU_control_ID;
•   wire [1:0] MemtoReg_ID , Jump_ID;
•   wire ALU_Src_B_ID , Branch_ID , BranchN_ID , MemRW_ID ,
    RegWrite_out_ID;
•
•   wire [31:0] PC_out_EX , Rs1_out_IDEX , Rs2_out_IDEX,
    Imm_out_IDEX;
•   wire [4:0] Rd_addr_out_IDEX;
•   wire [3:0] ALU_control_IDEX;
•   wire [1:0] MemtoReg_IDEX , Jump_IDEX;
•   wire ALUSrc_B_IDEX , Branch_IDEX , BranchN_IDEX ,
    MemRW_IDEX , RegWrite_IDEX;
•
•   wire [31:0] PC4_out_EX, ALU_out_EX, Rs2_out_EX;
•   wire zero_out_EX;
•
•   wire [31:0] PC_out_EXMem , PC4_out_EXMem ;
•   wire [4:0] Rd_addr_out_EXMem;
•   wire [1:0] MemtoReg_out_EXMem , Jump_out_EXMem;
•   wire zero_out_EXMem, Branch_out_EXMem, BranchN_out_EXMem,
    RegWrite_out_EXMem;
•
•   wire PCSrc;
•
•   wire [31:0] PC4_out_MemWB , ALU_out_MemWB,
    DMem_data_out_MemWB;
•   wire [4:0] Rd_addr_out_MemWB;
•   wire [1:0] MemtoReg_out_MemWB;
•   wire RegWrite_out_MemWB;

```

- 流水线 CPU 集成:

```

•   Pipeline_IF IF(
•       .clk_IF(clk),
•       .rst_IF(rst),
•       .en_IF(1'b1),
•       .PC_in_IF(PC_out_EXMem),
•       .PCSrc(PCSrc),
•       .PC_out_IF(PC_out_IF)
•   );
•   IF_reg_ID IF_reg_ID(

```

```

•         .clk_IFID(clk),
•         .rst_IFID(rst),
•         .en_IFID(1'b1),
•         .PC_in_IFID(PC_out_IF),
•         .inst_in_IFID(inst_IF),
•         .PC_out_IFID(PC_out_ID),
•         .inst_out_IFID(inst_ID)
•     );
•     Pipeline_ID ID(
•         .clk_ID(clk),
•         .rst_ID(rst),
•         .RegWrite_in_ID(RegWrite_out_MemWB),
•         .Rd_addr_ID(Rd_addr_out_MemWB),
•         .Wt_data_ID(Data_out_WB),
•         .Inst_in_ID(inst_ID),
•         .Rd_addr_out_ID(Rd_addr_out_ID),
•         .Rs1_out_ID(Rs1_out_ID),
•         .Rs2_out_ID(Rs2_out_ID),
•         .Imm_out_ID(Imm_out_ID),
•         .ALUSrc_B_ID(ALUSrc_B_ID),
•         .ALU_Control_ID(ALU_control_ID),
•         .Branch_ID(Branch_ID),
•         .BranchN_ID(BranchN_ID),
•         .MemRW_ID(MemRW_ID),
•         .Jump_ID(Jump_ID),
•         .MemtoReg_ID(MemtoReg_ID),
•         .RegWrite_out_ID(RegWrite_out_ID),
•         .all_regs_data(all_regs_data)
•     );
•     ID_reg_Ex ID_reg_Ex(
•         .clk_IDEX(clk),
•         .rst_IDEX(rst),
•         .en_IDEX(1'b1),
•         .PC_in_IDEX(PC_out_ID),
•         .Rd_addr_IDEX(Rd_addr_out_ID),
•         .Rs1_in_IDEX(Rs1_out_ID),
•         .Rs2_in_IDEX(Rs2_out_ID),
•         .Imm_in_IDEX(Imm_out_ID),
•         .ALUSrc_B_in_IDEX(ALUSrc_B_ID),
•         .ALU_control_in_IDEX(ALU_control_ID),
•         .Branch_in_IDEX(Branch_ID),
•         .BranchN_in_IDEX(BranchN_ID),
•         .MemRW_in_IDEX(MemRW_ID),
•         .Jump_in_IDEX(Jump_ID),

```

```

•      .MemtoReg_in_IDEX(MemtoReg_ID),
•      .RegWrite_in_IDEX(RegWrite_out_ID),
•      .PC_out_IDEX(PC_out_IDEX),
•      .Rd_addr_out_IDEX(Rd_addr_out_IDEX),
•      .Rs1_out_IDEX(Rs1_out_IDEX),
•      .Rs2_out_IDEX(Rs2_out_IDEX),
•      .Imm_out_IDEX(Imm_out_IDEX),
•      .ALUSrc_B_out_IDEX(ALUSrc_B_IDEX),
•      .ALU_control_out_IDEX(ALU_control_IDEX),
•      .Branch_out_IDEX(Branch_IDEX),
•      .BranchN_out_IDEX(BranchN_IDEX),
•      .MemRW_out_IDEX(MemRW_EX),
•      .Jump_out_IDEX(Jump_IDEX),
•      .MemtoReg_out_IDEX(MemtoReg_IDEX),
•      .RegWrite_out_IDEX(RegWrite_IDEX)
• );
• Pipeline_Ex Ex(
•      .PC_in_EX(PC_out_IDEX),
•      .Rs1_in_EX(Rs1_out_IDEX),
•      .Rs2_in_EX(Rs2_out_IDEX),
•      .Imm_in_EX(Imm_out_IDEX),
•      .ALUSrc_B_in_EX(ALUSrc_B_IDEX),
•      .ALU_control_in_EX(ALU_control_IDEX),
•      .PC_out_EX(PC_out_EX),
•      .PC4_out_EX(PC4_out_EX),
•      .zero_out_EX(zero_out_EX),
•      .ALU_out_EX(ALU_out_EX),
•      .Rs2_out_EX(Rs2_out_EX)
• );
• Ex_reg_Mem Ex_reg_Mem(
•      .clk_EXMem(clk),
•      .rst_EXMem(rst),
•      .en_EXMem(1'b1),
•      .PC_in_EXMem(PC_out_EX),
•      .PC4_in_EXMem(PC4_out_EX),
•      .Rd_addr_EXMem(Rd_addr_out_IDEX),
•      .zero_in_EXMem(zero_out_EX),
•      .ALU_in_EXMem(ALU_out_EX),
•      .Rs2_in_EXMem(Rs2_out_EX),
•      .Branch_in_EXMem(Branch_IDEX),
•      .BranchN_in_EXMem(BranchN_IDEX),
•      .MemRW_in_EXMem(MemRW_EX),
•      .Jump_in_EXMem(Jump_IDEX),
•      .MemtoReg_in_EXMem(MemtoReg_IDEX),

```

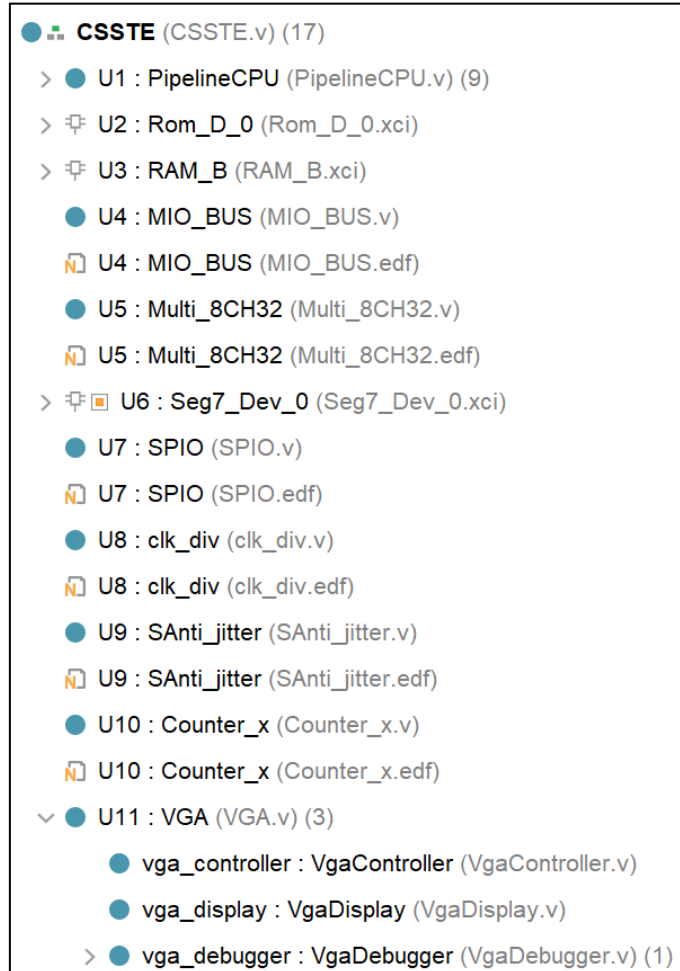
```

•      .RegWrite_in_EXMem(RegWrite_IDEX),
•      .PC_out_EXMem(PC_out_EXMem),
•      .PC4_out_EXMem(PC4_out_EXMem),
•      .Rd_addr_out_EXMem(Rd_addr_out_EXMem),
•      .zero_out_EXMem(zero_out_EXMem),
•      .ALU_out_EXMem(Addr_out),
•      .Rs2_out_EXMem(Data_out),
•      .Branch_out_EXMem(Branch_out_EXMem),
•      .BranchN_out_EXMem(BranchN_out_EXMem),
•      .MemRW_out_EXMem(MemRW_Mem),
•      .Jump_out_EXMem(Jump_out_EXMem),
•      .MemtoReg_out_EXMem(MemtoReg_out_EXMem),
•      .RegWrite_out_EXMem(RegWrite_out_EXMem)
• );
• Pipeline_Mem Mem(
•     .zero_in_Mem(zero_out_EXMem),
•     .Branch_in_Mem(Branch_out_EXMem),
•     .BranchN_in_Mem(BranchN_out_EXMem),
•     .Jump_in_Mem(Jump_out_EXMem),
•     .PCSrc(PCSrc) );
• Mem_reg_WB Mem_reg_WB(
•     .clk_MemWB(clk),
•     .rst_MemWB(rst),
•     .en_MemWB(1'b1),
•     .PC4_in_MemWB(PC4_out_EXMem),
•     .Rd_addr_MemWB(Rd_addr_out_EXMem),
•     .ALU_in_MemWB(Addr_out),
•     .DMem_data_MemWB(Data_in),
•     .MemtoReg_in_MemWB(MemtoReg_out_EXMem),
•     .RegWrite_in_MemWB(RegWrite_out_EXMem),
•     .PC4_out_MemWB(PC4_out_MemWB),
•     .Rd_addr_out_MemWB(Rd_addr_out_MemWB),
•     .ALU_out_MemWB(ALU_out_MemWB),
•     .DMem_data_out_MemWB(DMem_data_out_MemWB),
•     .MemtoReg_out_MemWB(MemtoReg_out_MemWB),
•     .RegWrite_out_MemWB(RegWrite_out_MemWB)
• );
• Pipeline_WB WB(
•     .PC4_in_WB(PC4_out_MemWB),
•     .ALU_in_WB(ALU_out_MemWB),
•     .DMem_data_WB(DMem_data_out_MemWB),
•     .MemtoReg_in_WB(MemtoReg_out_MemWB),
•     .Data_out_WB(Data_out_WB)
• );

```

3.2 建立流水线 SOC 工程:

Lab4 对单周期 CPU 进行了实现, 并设计了 SOC 工程 CSSTE.v 进行了相关物理验证, Lab5 也将进行物理验证, 因此需要对 CSSTE.v 进行微调 (附带整个工程结构图)。



```
module CSSTE(
    input clk_100mhz,
    input RSTN,
    input [3:0] BTN_y,
    input [15:0] SW,
    output [3:0] Blue,
    output [3:0] Green,
    output [3:0] Red,
    output HSYNC,
    output VSYNC,
    output [15:0] LED_out,
    output [7:0] segment,
    output [7:0] AN
);
```

```

wire [3:0] BTN_OK;
wire [15:0] SW_OK;
wire rst;
wire mem_w;
wire [31:0] Cpu_data2bus ,Cpu_data4bus;
wire [31:0] addr;
wire [31:0] ram_data_out;
wire [31:0] counter_out;
wire counter0_out;
wire counter1_out;
wire counter2_out;
wire [31:0] ram_data_in;
wire [9:0] ram_addr;
wire data_ram_we;
wire GPIOf0000000_we;
wire GPIOe0000000_we;
wire counter_we;
wire [31:0] Peripheral_in;
wire [31:0] clk_div;
wire clk_CPU;
wire [1:0] counter_set;
wire [63:0] point_in;
wire [31:0]
wire [31:0]
PC_out_IF,PC_out_ID,PC_out_Ex,inst_ID,inst_IF,Data_out_WB;
wire MemRW_EX , MemRW_Mem ;
wire [1023:0] all_regs_data;

assign point_in = {clk_div,clk_div};
PipelineCPU U1(
    .clk(clk_CPU),
    .rst(rst),
    .Data_in(Cpu_data4bus),
    .PC_out_IF(PC_out_IF[31:0]),
    .inst_IF(inst_IF),
    .PC_out_ID(PC_out_ID[31:0]),
    .inst_ID(inst_ID),
    .PC_out_IDEX(PC_out_Ex),
    .MemRW_EX(MemRW_EX),
    .MemRW_Mem(MemRW_Mem),
    .Data_out(Cpu_data2bus),
    .Addr_out(addr),
    .Data_out_WB(Data_out_WB),
    .all_regs_data(all_regs_data));

```

```

Rom_D_0 U2 (
    .a(PC_out_IF[11:2]),
    .spo(inst_IF)
);

wire [31:0] RAM_B_douta;
RAM_B U3(
    .addra(ram_addr),
    .clka(~clk_100mhz),
    .dina(ram_data_in),
    .douta(RAM_B_douta), // U4 & U11
    .wea(data_ram_we)
);

MIO_BUS U4(
    .clk(clk_100mhz),
    .rst(rst),
    .BTN(BTN_OK[3:0]),
    .SW(SW_OK[15:0]),
    .mem_w(MemRW_Mem),
    .Cpu_data2bus(Cpu_data2bus[31:0]),
    .addr_bus(addr[31:0]),
    .ram_data_out(RAM_B_douta[31:0]),
    .led_out(LED_out[15:0]), //From U7
    .counter_out(counter_out[31:0]), // From U10
    .counter0_out(counter0_out),
    .counter1_out(counter1_out),
    .counter2_out(counter2_out), // all U10
    .Cpu_data4bus(Cpu_data4bus[31:0]),
    .ram_data_in(ram_data_in[31:0]),
    .ram_addr(ram_addr[9:0]),
    .data_ram_we(data_ram_we),
    .GPIOf0000000_we(GPIOf0000000_w),
    .GPIOe0000000_we(GPIOe0000000_we),
    .counter_we(counter_we),
    .Peripheral_in(Peripheral_in[31:0])
);

wire [7:0] point_out;
wire [7:0] le_out;
wire [31:0] disp_num;

Multi_8CH32 U5(
    .clk(~clk_CPU),

```

```

        .rst(rst),
        .EN(GPIOe0000000_we),
        .point_in(point_in),
        .LES(64'b0),
        .Test(SW_OK[7:5]),
        .Data0(Peripheral_in),
        .data1({2'b00,PC_out_IF[31:2]}),
        .data2(inst_IF),
        .data3(counter_out),
        .data4(addr),
        .data5(Cpu_data2bus),
        .data6(Cpu_data4bus),
        .data7(PC_out_IF),
        .point_out(point_out),
        .LE_out(le_out),
        .Disp_num(dis_num)
    );
    Seg7_Dev_0 U6(
        .disp_num(dis_num),
        .point(point_out),
        .les(le_out),
        .scan({clk_div[18],clk_div[17],clk_div[16]}),
        .AN(AN),
        .segment(segment)
    );
    SPIO U7(
        .clk(~clk_CPU),
        .rst(rst),
        .Start(clk_div[20]),
        .EN(GPIOf0000000_we),
        .P_Data(Peripheral_in),
        .counter_set(counter_set),
        .LED_out(LED_out)
    );

    clk_div U8(
        .clk(clk_100mhz),
        .rst(rst),
        .SW2(SW_OK[2]),
        .SW8(SW_OK[8]),
        .STEP(SW_OK[10]),
        .clkdiv(clk_div),
        .Clk_CPU(clk_CPU)
    );

```

```

SAnti_jitter U9(
    .clk(clk_100mhz),
    .RSTN(RSTN),
    .Key_y(BTN_y),
    .SW(SW[15:0]),
    .BTN_OK(BTN_OK[3:0]),
    .SW_OK(SW_OK[15:0]),
    .rst(rst)
);

Counter_x U10(
    .clk(~clk_CPU),
    .rst(rst),
    .clk0(clk_div[6]),
    .clk1(clk_div[9]),
    .clk2(clk_div[11]),
    .counter0_OUT(counter0_out),
    .counter1_OUT(counter1_out),
    .counter2_OUT(counter2_out),
    .counter_we(counter_we),
    .counter_out(counter_out),
    .counter_val(Peripheral_in),
    .counter_ch(counter_set)
);

VGA U11(
    .clk_25m(clk_div[1]),
    .clk_100m(clk_100mhz),
    .rst(rst),
    .PC_IF(PC_out_IF[31:0]),
    .PC_ID(PC_out_ID[31:0]),
    .inst_IF(inst_IF),
    .inst_ID(inst_ID),
    .PC_Ex(PC_out_Ex),
    .MemRW_Ex(MemRW_EX),
    .MemRW_Mem(MemRW_Mem),
    .Data_out(Cpu_data2bus),
    .Addr_out(addr),
    .Data_out_WB(Data_out_WB),
    .hs(HSYNC),
    .vs(VSYNC),
    .vga_r(Red),
    .vga_g(Green),
    .vga_b(Blue),
    .all_regs_data(all_regs_data) );

```

```
endmodule
```

四、实验结果分析

```
PipelineCPU U1(  
    .clk(clk_CPU),  
    .rst(rst),  
    .Data_in(Cpu_data4bus),  
    .PC_out_IF(PC_out_IF[31:0]),  
    .inst_IF(inst_IF),  
    .PC_out_ID(PC_out_ID[31:0]),  
    .inst_ID(inst_ID),  
    .PC_out_IDEX(PC_out_Ex),  
    .MemRW_EX(MemRW_EX),  
    .MemRW_Mem(MemRW_Mem),  
    .Data_out(Cpu_data2bus),  
    .Addr_out(addr),  
    .Data_out_WB(Data_out_WB),  
    .all_regs_data(all_regs_data)  
);  
  
Rom_D_0 U2 (  
    .a(PC_out_IF[11:2]),  
    .spo(inst_IF)  
);  
  
wire [31:0] RAM_B_douta;  
  
RAM_B U3(  
    .addra(ram_addr),  
    .clka(~clk_100mhz),  
    .dina(ram_data_in),  
    .douta(RAM_B_douta), // U4 & U11  
    .wea(data_ram_we)  
);  
  
MIO_BUS U4(  
    .clk(clk_100mhz),  
    .rst(rst),  
    .BTN(BTN_OK[3:0]),  
    .SW(SW_OK[15:0]),  
    .mem_w(MemRW_Mem),  
    .Cpu_data2bus(Cpu_data2bus[31:0]),  
    .addr_bus(addr[31:0]),  
    .ram_data_out(RAM_B_douta[31:0]),  
    .led_out(LED_out[15:0]), //From U7  
    .counter_out(counter_out[31:0]), // From U10  
    .counter0_out(counter0_out),  
    .counter1_out(counter1_out),  
    .counter2_out(counter2_out), // all U10  
    .Cpu_data4bus(Cpu_data4bus[31:0]),
```

Lab5 由于设计的 Pipeline_CPU 输入需要来自 MIO_BUS (.edf 文件) 的信号 Cpu_data4bus、Cpu_data2bus 等，但对 PipelineCPU 进行仿真平台的构建时可以将 Cpu_data4bus 直接替换为 RAM_B_douta 进行连续指令仿真(Lab5-3 时仿真)。

同时为了实验报告内容尽量不重复，物理验证统一在 Lab5-2~3 自主实现流水线各阶段代码并且替换.edf 文件后再进行物理验证。

五、讨论与心得

单纯的连线操作，但由于流水线 CPU 较复杂且提供的文件中有较多小错误，需要仔细检查，不然后期实验 debug 很痛苦。

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 14 日

浙江大学实验报告

课程名称： 计算机组成与设计 实验类型： 综合

实验项目名称： 流水线处理器—IF、ID 设计与集成

学生姓名： 张序鸿 专业： 混合班 学号： 3230103265

指导老师： 杨莹春、潘之杰 助教： 马致远

实验地点： 东 4-509 实验日期： 2024 年 12 月 11 日

一、实验目的和要求

- 1.理解流水线 CPU 的基本原理和组织结构
- 2.掌握五级流水线的工作过程和设计方法
- 3.理解流水线取指、译码的设计原理
- 4.设计流水线测试程序

二、实验内容和原理

内容：

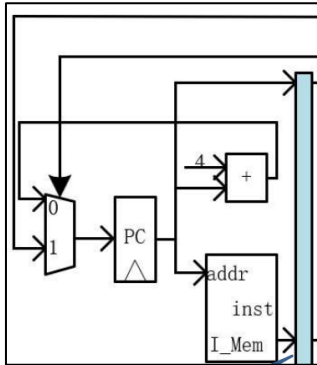
任务一：设计取指（IF）、译码（ID）模块

- 设计取指模块，替换 OExp05-1 的取指模块并完成集成
- 设计译码模块，替换 OExp05-1 的译码模块并完成集成

任务二：设计 CPU 中断测试方案并完成测试

原理:

2.1 取值 IF 部件



IF: 取指阶段涉及程序计数器 (PC) 和指令存储器 (I_Mem), 程序计数器输出作为地址从指令存储器中读取指令。

IF_reg_ID: 暂存指令、PC, 以待下一级使用。

(1) 程序计数器 (Program Counter) :

一个 32 位的寄存器, 用于锁存地址, 其输出作为访存地址读指令存储器, 访存结果为当前指令内容。

```
module REG32(
    input clk,
    input rst,
    input CE,
    input [31:0]D,
    output reg [31:0]Q
);

always @(posedge clk or posedge rst)
    if (rst==1)
        Q <= 32'b0;
    else if (CE == 1)
        Q <= D;
endmodule
```

(2) 加法器 (add):

一个输入 32 位的加法器, 用于计算 PC+4(顺序执行程序时新的 PC 值)。

```
`timescale 1ns / 1ps
module add32(
    input [31:0] a,
    input [31:0] b,
    output [31:0] c
);

assign c = a + b;
endmodule
```

(3) 32 位多路选择器:

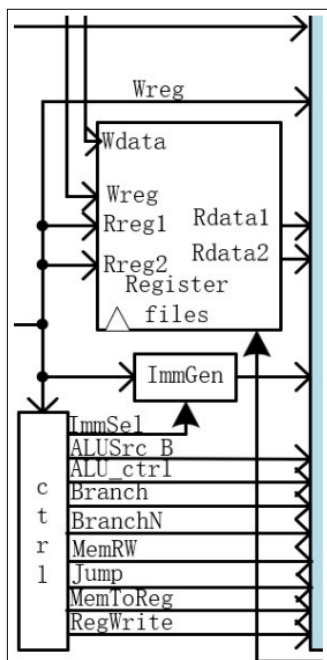
一个 32 位二选一的多路器，用于选择 PC 的更新值，当 PCSrc = 0，选择 PC+4；当 PCSrc = 1，选择跳转目标地址。

```
module MUX2T1_32(  
    input [31:0] I0,  
    input [31:0] I1,  
    input s, // 此处输入 s = PCSrc  
    output [31:0] o  
);  
  
    assign o = s ? I1 : I0;  
endmodule
```

(4) 指令存储器 (I_Mem):

一个异步 ROM，输入地址则输出数据，用于存储 CPU 的指令。在 IF 部件设计时不需要添加，只需要在构建 SOC 时添加。

2.2 译码 ID 部件



ID: 译码阶段涉及寄存器堆 (RegisterFiles) 和译码器 (SCPU_ctrl)、立即数生成单元 (ImmGen)。

从寄存器堆可以读取操作数后，译码器对指令进行解析产生各种各种控制信号，立即数生成单元根据控制信号和输入指令生成各种类型的立即数。

ID_reg_Ex: 暂存 PC 值，寄存器读取的数据 (rs1, rs2)，立即数和控制信号以待下一级使用

(1) 寄存器堆 (RegsFiles):

本质是 32×32bit 寄存器组，指令操作的主要对象；根据寄存器号读取和写入数据，带有写 控制信号。

```

module Regs(
    input clk,
    input rst,
    input [4:0] Rs1_addr,
    input [4:0] Rs2_addr,
    input [4:0] Wt_addr,
    input [31:0]Wt_data,
    input RegWrite,
    output [31:0] Rs1_data,
    output [31:0] Rs2_data,
    output [1023:0] all_regs_data
);
reg [31:0] register [1:31];
integer i;

assign Rs1_data = (Rs1_addr== 0) ? 0 : register[Rs1_addr];
assign Rs2_data = (Rs2_addr== 0) ? 0 : register[Rs2_addr];

always @(posedge clk or posedge rst) begin
    if (rst==1)
        for (i=1; i<32; i=i+1)
            register[i] <= 0;
    else if ((Wt_addr != 0) && (RegWrite == 1))
        register[Wt_addr] <= Wt_data;
end
endmodule

```

(2) 立即数生成单元 (ImmGen):

根据输入指令 (ImmSel) 生成立即数，此处采用拓展后的 ImmGen.

```

module ImmGen_more(
    input wire [2:0] ImmSel,
    input wire [31:0] inst_field,
    output reg [31:0] Imm_out
);

always@*begin
    case(ImmSel)
        3'b000:Imm_out = {inst_field[31:12],12'h000}; // U-type
        3'b001:Imm_out = {{20{inst_field[31]}},inst_field[31:20]};
        //addi\lw(I)
        3'b010:Imm_out =
        {{20{inst_field[31]}},inst_field[31:25],inst_field[11:7]}; //sw(s)
    endcase
end

```

```

        3'b011:Imm_out =
{{20{inst_field[31]}},inst_field[7],inst_field[30:25],inst_field[11:8],
1'b0}; //beq(b)
        3'b100:Imm_out =
{{12{inst_field[31]}},inst_field[19:12],inst_field[20],inst_field[30:21
],1'b0}; //jal(j)
        default: Imm_out = 32'h00000004;
    endcase
end
endmodule

```

(3) 译码控制器 (SCPU_ctrl):

根据输入指令译码输出各个部件的控制信号（指令拓展后版本）。

```

module SCPU_ctrl_more(
    input[4:0]OPcode, //Opcode
    input[2:0]Fun3, //Function
    input Fun7, //Function
    input MIO_ready, //CPU Wait

    output reg [2:0]ImmSel, //立即数选择控制
    output reg ALUSrc_B, //源操作数 2 选择
    output reg [1:0]MemtoReg, //写回数据选择控制
    output reg [1:0]Jump, //jal
    output reg Branch, //beq
    output reg BranchN,
    output reg RegWrite, //寄存器写使能
    output reg MemRW, //存储器读写使能
    output reg [3:0]ALU_Control, //alu 控制
    output reg CPU_MIO
);
reg [1:0] ALUop;
wire [3:0] Fun;
assign Fun = {Fun3, Fun7};
always @(*) begin
    case(OPcode)
        5'b01100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_1_0_0_0_00_10_000; end //ALU
        5'b00000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_01_1_0_0_0_00_00_001; end //load
        5'b01000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_0_1_0_0_00_00_010; end //store

```

```

        5'b11000: begin
            case(Fun3)
                3'b000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_1_0_00_01_011; end // beq
                3'b001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_0_1_00_01_011; end // bne
            endcase
        end

        5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_01_00_100; end //jump
        5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_1_0_0_0_00_11_001; end //ALU(addi;;;)
        5'b11001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_10_00_001; end //jarl
        5'b01101: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_11_1_0_0_0_00_00_000; end //lui
        default:
begin {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} = 14'h0; end
        endcase
    end

    always @(*) begin
        case(ALUop)
            2'b00: ALU_Control = 4'b0010 ;
            2'b01: ALU_Control = 4'b0110 ;
            2'b10:
                case(Fun)
                    4'b0000: ALU_Control = 4'b0010 ; //add
                    4'b0001: ALU_Control = 4'b0110 ; //sub
                    4'b1110: ALU_Control = 4'b0000 ; //and
                    4'b1100: ALU_Control = 4'b0001 ; //or
                    4'b0100: ALU_Control = 4'b0111 ; //slt
                    4'b1010: ALU_Control = 4'b1101 ; //srl
                    4'b1000: ALU_Control = 4'b1100 ; //xor
                    4'b0110: ALU_Control = 4'b1001 ; //sltu
                    4'b0010: ALU_Control = 4'b1110 ; //sll
                    4'b1011: ALU_Control = 4'b1111 ; //sra

```

```

        default: ALU_Control = 4'bx;
    endcase
2'b11:
    case(Fun3)
        3'b000: ALU_Control = 4'b0010 ; //addi
        3'b010: ALU_Control = 4'b0111 ; //slti
        3'b100: ALU_Control = 4'b1100 ; //xori
        3'b110: ALU_Control = 4'b0001 ; //ori
        3'b111: ALU_Control = 4'b0000 ; //andi
        3'b101:
            if(Fun7)
                ALU_Control = 4'b1111 ; //srai
            else
                ALU_Control = 4'b1101 ; //srli
        3'b011: ALU_Control = 4'b1001 ; //sltiu
        3'b001: ALU_Control = 4'b1110 ; //slli
        default: ALU_Control = 4'bx ;
    endcase
endcase
end
endmodule

```

三、实验过程和数据记录

3.1 设计实现取值（IF）模块：

① Pipeline_IF :

PipelineCPU 第一阶段根据 PC 从指令存储器中取出指令并进行 PC 更新。



```
`timescale 1ns / 1ps
module Pipeline_IF(
    input clk_IF,
    input rst_IF,
    input en_IF,
    input [31:0]PC_in_IF,
    input PCSrc,
    output [31:0]PC_out_IF
);

wire [31:0] U0_out , PC_in;
MUX2T1_32 U0(
    .s(PCSrc),
    .I0(PC_in),
    .I1(PC_in_IF),
    .o(U0_out)
);

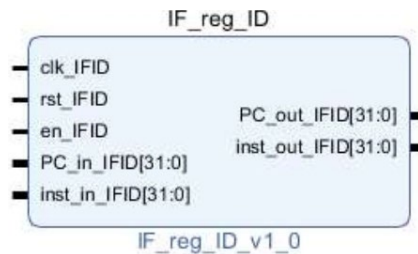
REG32 PC(
    .clk(clk_IF),
    .rst(rst_IF),
    .CE(en_IF),
    .D(U0_out),
    .Q(PC_out_IF)
);

add32 add(
    .a(PC_out_IF),
    .b(32'h4),
    .c(PC_in)
);

endmodule
```

② IF_reg_ID :

PipelineCPU 取值和译码之间的寄存器，寄存 IF 级的输出指令，分割 IF 级和 ID 级的指令或控制信号，防止相互干扰，在 IF 级执行结束时将指令的控制信号传递至下一级。



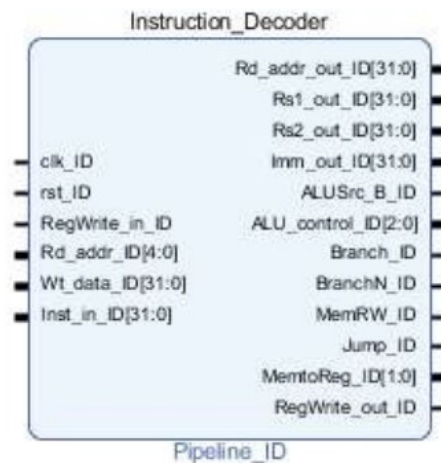
```
`timescale 1ns / 1ps
module IF_reg_ID(
    input clk_IFID,
    input rst_IFID,
    input en_IFID,
    input [31:0]PC_in_IFID,
    input [31:0]inst_in_IFID,
    output reg [31:0]PC_out_IFID,
    output reg [31:0]inst_out_IFID
);

    always @(posedge clk_IFID or posedge rst_IFID)
        if (rst_IFID == 1'b1) begin
            PC_out_IFID <= 32'h0;
            inst_out_IFID <= 32'h0;
        end
        else if (en_IFID == 1'b1) begin
            PC_out_IFID <= PC_in_IFID;
            inst_out_IFID <= inst_in_IFID;
        end
    end
endmodule
```

3.2 设计实现译码（ID）模块：

① Pipeline_ID :

PipelineCPU 第二阶段，通过 SCPU_ctrl 译码之后产生各种控制信号，同时寄存器堆根据所需操作数寄存器的索引读出操作数，立即数生成单元输出所需立即数。



```
`timescale 1ns / 1ps

module Pipeline_ID(
    input clk_ID,
    input rst_ID,
    input RegWrite_in_ID,
    input [4:0]Rd_addr_ID,
    input [31:0]Wt_data_ID,
    input [31:0]Inst_in_ID,

    output [4:0]Rd_addr_out_ID,
    output [31:0]Rs1_out_ID,
    output [31:0]Rs2_out_ID,
    output [31:0]Imm_out_ID,
    output ALUSrc_B_ID,
    output [3:0]ALU_Control_ID,
    output Branch_ID,
    output BranchN_ID,
    output MemRW_ID,
    output [1:0]Jump_ID,
    output [1:0]MemtoReg_ID,
    output RegWrite_out_ID,
    output [1023:0] all_regs_data
);
```

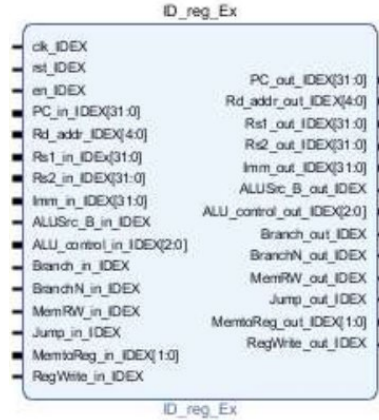
```

wire[2:0] ImmSel;
assign Rd_addr_out_ID = Inst_in_ID[11:7];
Regs Regs_0(
    .clk(clk_ID),
    .rst(rst_ID),
    .Rs1_addr(Inst_in_ID[19:15]),
    .Rs2_addr(Inst_in_ID[24:20]),
    .Wt_addr(Rd_addr_ID),
    .Wt_data(Wt_data_ID),
    .RegWrite(RegWrite_in_ID),
    .Rs1_data(Rs1_out_ID),
    .Rs2_data(Rs2_out_ID),
    .all_regs_data(all_regs_data)
);
ImmGen_more ImmGen_0(
    .ImmSel(ImmSel),
    .inst_field(Inst_in_ID),
    .Imm_out(Imm_out_ID)
);
SCPU_ctrl_more SCPU_ctrl_0(
    .OPcode(Inst_in_ID[6:2]),
    .Fun3(Inst_in_ID[14:12]),
    .Fun7(Inst_in_ID[30]),
    .ImmSel(ImmSel),
    .ALUSrc_B(ALUSrc_B_ID),
    .MemtoReg(MemtoReg_ID),
    .Jump(Jump_ID),
    .Branch(Branch_ID),
    .BranchN(BranchN_ID),
    .RegWrite(RegWrite_out_ID),
    .MemRW(MemRW_ID),
    .ALU_Control(ALU_Control_ID)
);
endmodule

```

② ID_reg_Ex :

PipelineCPU 译码和执行之间的寄存器，存储 ALU 数据和控制信号。寄存 ID 级的输出指令，分割 ID 级和 EX 级的指令或控制信号，防止相互干扰，在 ID 级执行结束时将指令的控制信号传递至下一级。



```
`timescale 1ns / 1ps

module ID_reg_Ex(
    input clk_IDEX, //寄存器时钟
    input rst_IDEX, //寄存器复位
    input en_IDEX, //寄存器使能
    input[31:0] PC_in_IDEX, //PC 输入
    input[4:0] Rd_addr_IDEX, //写目的地址输入
    input[31:0] Rs1_in_IDEX, //操作数 1 输入
    input[31:0] Rs2_in_IDEX, //操作数 2 输入
    input[31:0] Imm_in_IDEX, //立即数输入
    input ALUSrc_B_in_IDEX, //ALU B 输入选择
    input[3:0] ALU_control_in_IDEX, //ALU 选择控制
    input Branch_in_IDEX, //Beq
    input BranchN_in_IDEX, //Bne
    input MemRW_in_IDEX, //存储器读写
    input[1:0] Jump_in_IDEX, //Jal
    input[1:0] MemtoReg_in_IDEX, //写回选择
    input RegWrite_in_IDEX, //寄存器堆读写

    output reg[31:0] PC_out_IDEX, //PC 输出
    output reg[4:0] Rd_addr_out_IDEX, //目的地址输出
    output reg[31:0] Rs1_out_IDEX, //操作数 1 输出
    output reg[31:0] Rs2_out_IDEX, //操作数 2 输出
    output reg[31:0] Imm_out_IDEX, //立即数输出
    output reg ALUSrc_B_out_IDEX, //ALU B 选择
    output reg[3:0] ALU_control_out_IDEX, //ALU 控制
    output reg Branch_out_IDEX, //Beq
```

```

output reg BranchN_out_IDEX, //Bne
output reg MemRW_out_IDEX, //存储器读写
output reg[1:0]Jump_out_IDEX, //Jal
output reg[1:0] MemtoReg_out_IDEX, //写回
output reg RegWrite_out_IDEX //寄存器堆读写
);

always @(posedge clk_IDEX or posedge rst_IDEX) begin
    if(rst_IDEX == 1'b1)begin
        PC_out_IDEX <= 32'h0;
        Rd_addr_out_IDEX <= 5'h0;
        Rs1_out_IDEX <= 32'h0;
        Rs2_out_IDEX <= 32'h0;
        Imm_out_IDEX <= 32'h0;
        ALUSrc_B_out_IDEX <= 1'h0;
        ALU_control_out_IDEX <= 4'h0;
        Branch_out_IDEX <= 1'h0;
        BranchN_out_IDEX <= 1'h0;
        MemRW_out_IDEX <= 1'h0;
        Jump_out_IDEX <= 2'h0;
        MemtoReg_out_IDEX <= 2'h0;
        RegWrite_out_IDEX <= 1'h0;
    end
    else if(en_IDEX == 1'b1) begin
        PC_out_IDEX <= PC_in_IDEX;
        Rd_addr_out_IDEX <= Rd_addr_IDEX;
        Rs1_out_IDEX <= Rs1_in_IDEX;
        Rs2_out_IDEX <= Rs2_in_IDEX;
        Imm_out_IDEX <= Imm_in_IDEX;
        ALUSrc_B_out_IDEX <= ALUSrc_B_in_IDEX;
        ALU_control_out_IDEX <= ALU_control_in_IDEX;
        Branch_out_IDEX <= Branch_in_IDEX;
        BranchN_out_IDEX <= BranchN_in_IDEX;
        MemRW_out_IDEX <= MemRW_in_IDEX;
        Jump_out_IDEX <= Jump_in_IDEX;
        MemtoReg_out_IDEX <= MemtoReg_in_IDEX;
        RegWrite_out_IDEX <= RegWrite_in_IDEX;
    end
end

endmodule

```

四、实验结果分析

~暂无~

五、讨论与心得

原 ppt 的 ID 部分未采用指令拓展,仅添加了 BranchN 信号来实现 bne 指令。为此,我干脆使用了指令拓展后的 ImmGen 和 SCPU_ctrl,并同时修改了 ALU_control、Jump、ImmSel 信号,实现统一。

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 13 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: 流水线处理器—Ex、Mem、WB 设计与集成

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 马致远

实验地点: 东 4-509 实验日期: 2024 年 12 月 11 日

一、实验目的和要求

- 1.理解流水线 CPU 的基本原理和组织结构
- 2.掌握五级流水线的工作过程和设计方法
- 3.理解流水线执行、存储器访问、写回的原理
- 4.设计流水线测试程序

二、实验内容和原理

内容:

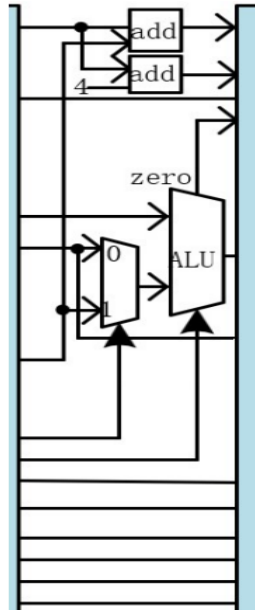
任务一: 设计执行 (Ex)、存储器访问 (Mem)、写回 (WB) 模块

- 设计执行模块, 替换 OExp05-2 的执行模块并完成集成
- 设计访存模块, 替换 OExp05-2 的访存模块并完成集成
- 设计写回模块, 替换 OExp05-2 的写回模块并完成集成

任务二: 设计流水线测试方案并完成测试

原理:

2.1 执行 Ex 部件



Ex : 执行阶段涉及运算单元 (ALU) 它获取操作数并完成指定的算数运算或逻辑运算。

Ex_reg_Mem: 暂存运算结果和控制信号, 以待下一级使用。

(1) 32 位多路选择器:

本质是一个 32 位二选一的多路器, 用于选择 ALU 的第二个操作数, 当 $ALUSrc_B = 0$, 选择寄存器堆数据 2; 当 $ALUSrc_B = 1$, 选择立即数。

```
module MUX2T1_32(  
    input [31:0] I0,  
    input [31:0] I1,  
    input s, // 此处输入 s = ALUSrc_B  
    output [31:0] o  
);  
  
    assign o = s ? I1 : I0;  
endmodule
```

(2) 加法器 (add):

本质是一个 32 位输入的加法器, 一个输出计算 $PC+4$ 、一个输出计算 $PC+Imm$ 。

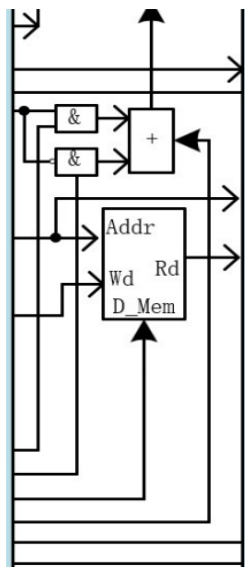
```
`timescale 1ns / 1ps  
module add32(  
    input [31:0] a,  
    input [31:0] b,  
    output [31:0] c  
);  
  
    assign c = a + b;  
endmodule
```

(3) ALU:

本质是一个二输入 32 位的运算器，实现算数运算和逻辑运算

```
module ALU(  
    input [31:0] A,  
    input [3:0] ALU_operation, // 4 位采用了指令拓展  
    input [31:0] B,  
    output reg [31:0] res,  
    output zero  
);  
always @ (*)  
    case (ALU_operation)  
        4'b0000: res = A & B; //and  
        4'b0001: res = A | B; //or  
        4'b0010: res = A + B; //add  
        4'b1100: res = A ^ B; //xor  
        4'b1101: res = A >> B; //srl  
        4'b0110: res = A - B; //sub  
        4'b0111: res = ($signed(A) < $signed(B)) ? 32'h1 : 32'h0; //slt  
        4'b1111: res = ($signed(A) >> B); //sra  
        4'b1110: res = A << B; //sll  
        4'b1001: res = (A < B) ? 32'h1 : 32'h0; //sltu  
        default: res = 32'hx;  
    endcase  
  
    assign zero = (res == 0) ? 1 : 0;  
endmodule
```

2.2 访存 Mem 部件



Mem： 存储器访问阶段涉及数据存储器 (D_Mem)，Load\Store 指令对数据存储器进行读或写。

Mem_reg_WB： 暂存存储器结果 和控制信号，以待下一级使用

(1) PCSrc 控制:

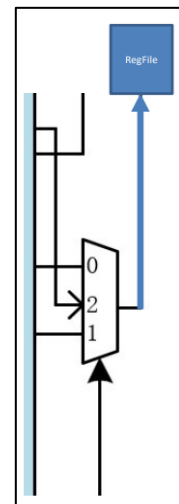
两输入与门和四输入或门 (beq、bne、jal、jalr) 组成, 用于判断分支和跳转是否发生, 生成 PCSrc 选择控制 PC 值。

(2) 数据储存器 (D_Mem):

本质是一个 RAM (可读可写), LOAD、STORE 指令的操作对象。同样的, 这只在构建 SOC 系统是用到。

2.3 写回 WB 部件

写回阶段涉及寄存器堆 (RegisterFiles), 将 ALU 的运算结果、存储器输出结果、PC+4 写回到寄存器堆。



(1) 4 路多路选择器:

本质是一个 32 位四选一的多路器, 用于选择写回寄存器堆的数据。

- 当 MemtoReg=0; 选择 ALU 输出,
- 当 MemtoReg=1; 选择存储器 (Memory) 输出
- 当 MemtoReg=2; 选择 PC+4。

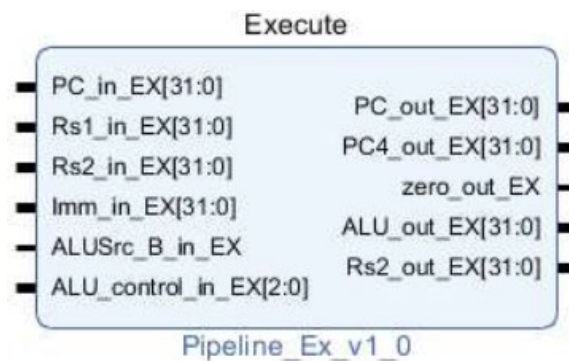
```
module MUX4T1_32(  
    input [31:0] I0,  
    input [31:0] I1,  
    input [31:0] I2,  
    input [31:0] I3,  
    input [1:0] s,  
    output reg [31:0] o  
);  
always @ (*) begin  
    case(s)  
        2'b00: o <= I0;  
        2'b01: o <= I1;  
        2'b10: o <= I2;  
        2'b11: o <= I3;  
    endcase  
end  
endmodule
```

三、实验过程和数据记录

3.1 设计实现执行（Ex）模块：

① Pipeline_Ex :

PipelineCPU 第三阶段，执行指令所指定的操作（ALU 计算、对 PC 进行 +4 或加立即数处理）。



```
`timescale 1ns / 1ps
module Pipeline_Ex(
    input[31:0] PC_in_EX, //PC 输入
    input[31:0] Rs1_in_EX, //操作数 1 输入
    input[31:0] Rs2_in_EX, //操作数 2 输入
    input[31:0] Imm_in_EX , //立即数输入
    input ALUSrc_B_in_EX , //ALU B 选择
    input[3:0] ALU_control_in_EX, //ALU 选择控制
    output [31:0] PC_out_EX, //PC 输出
    output [31:0] PC4_out_EX, //PC+4 输出
    output zero_out_EX, //ALU 判 0 输出
    output [31:0] ALU_out_EX, //ALU 计算输出
    output [31:0] Rs2_out_EX //操作数 2 输出
);

wire[31:0] mux_out;
assign Rs2_out_EX = Rs2_in_EX;

add32 add_32_0(
    .a(PC_in_EX),
    .b(Imm_in_EX),
    .c(PC_out_EX)
);

add32 add_32_1(
    .a(32'h4),
```

```

        .b(PC_in_EX),
        .c(PC4_out_EX)
    );

    ALU ALU(
        .A(Rs1_in_EX),
        .ALU_operation(ALU_control_in_EX),
        .B(mux_out),
        .res(ALU_out_EX),
        .zero(zero_out_EX)
    );

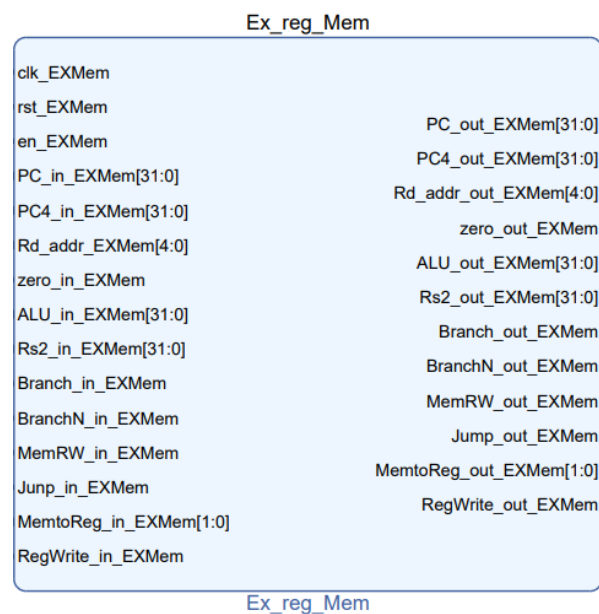
    MUX2T1_32 MUX2T1_32_0(
        .I0(Rs2_in_EX),
        .I1(Imm_in_EX),
        .s(ALUSrc_B_in_EX),
        .o(mux_out)
    );

endmodule

```

② Ex_reg_Mem :

PipelineCPU 执行和访存之间的寄存器，存储 ALU 数据和控制信号。寄存 EX 级的输出指令，分割 EX 级和 MEM 级的指令或控制信号，防止相互干扰，在 EX 级执行结束时将指令的控制信号传递至下一级。



```

`timescale 1ns / 1ps

module Ex_reg_Mem(
    input clk_EXMem,
    input rst_EXMem,
    input en_EXMem,
    input [31:0]PC_in_EXMem,
    input [31:0]PC4_in_EXMem,
    input [4:0]Rd_addr_EXMem,
    input zero_in_EXMem,
    input [31:0]ALU_in_EXMem,
    input [31:0]Rs2_in_EXMem,
    input Branch_in_EXMem,
    input BranchN_in_EXMem,
    input MemRW_in_EXMem,
    input [1:0]Jump_in_EXMem,
    input [1:0]MemtoReg_in_EXMem,
    input RegWrite_in_EXMem,

    output reg[31:0]PC_out_EXMem,
    output reg[31:0]PC4_out_EXMem,
    output reg[4:0]Rd_addr_out_EXMem,
    output reg zero_out_EXMem,
    output reg[31:0]ALU_out_EXMem,
    output reg[31:0]Rs2_out_EXMem,
    output reg Branch_out_EXMem,
    output reg BranchN_out_EXMem,
    output reg MemRW_out_EXMem,
    output reg[1:0]Jump_out_EXMem,
    output reg[1:0]MemtoReg_out_EXMem,
    output reg RegWrite_out_EXMem
);

always @(posedge clk_EXMem or posedge rst_EXMem)
begin
    if(rst_EXMem == 1'b1) begin
        PC_out_EXMem <= 32'h0;
        PC4_out_EXMem <= 32'h0;
        Rd_addr_out_EXMem <= 5'h0;
        zero_out_EXMem <= 1'h0;
        ALU_out_EXMem <= 32'h0;
        Rs2_out_EXMem <= 32'h0;
        Branch_out_EXMem <= 1'h0;
        BranchN_out_EXMem <= 1'h0;
    end
end

```

```

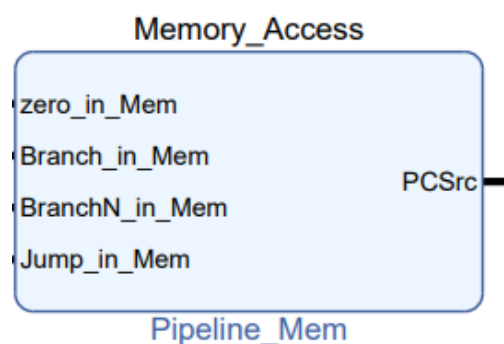
        MemRW_out_EXMem <= 1'h0;
        Jump_out_EXMem <= 2'h0;
        MemtoReg_out_EXMem <= 2'h0;
        RegWrite_out_EXMem <= 1'h0;
    end
    else if(en_EXMem == 1'b1) begin
        PC_out_EXMem <= PC_in_EXMem;
        PC4_out_EXMem <= PC4_in_EXMem;
        Rd_addr_out_EXMem <= Rd_addr_EXMem;
        zero_out_EXMem <= zero_in_EXMem;
        ALU_out_EXMem <= ALU_in_EXMem;
        Rs2_out_EXMem <= Rs2_in_EXMem;
        Branch_out_EXMem <= Branch_in_EXMem;
        BranchN_out_EXMem <= BranchN_in_EXMem;
        MemRW_out_EXMem <= MemRW_in_EXMem;
        Jump_out_EXMem <= Jump_in_EXMem;
        MemtoReg_out_EXMem <= MemtoReg_in_EXMem;
        RegWrite_out_EXMem <= RegWrite_in_EXMem;
    end
end
endmodule

```

3.2 设计实现访存（Mem）模块：

① Pipeline_Mem :

PipelineCPU 第四阶段，访问存储器，从 Memory 中读出数据或写入 Memory 的过程。



此处的输出 PCSrc 用于判断是否发生了指令跳转（如 beq、bne、jal 等），并作为 Pipeline_IF 的输入信号，从而获得下一条指令的地址。

```

`timescale 1ns / 1ps

module Pipeline_Mem(
    input zero_in_Mem,
    input Branch_in_Mem,
    input BranchN_in_Mem,
    input [1:0]Jump_in_Mem,

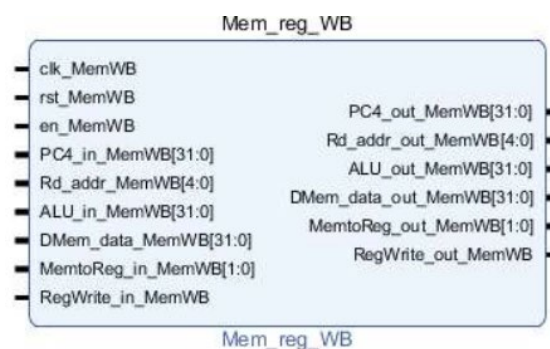
    output PCSrc
);

    assign PCSrc = (Branch_in_Mem & zero_in_Mem) | (BranchN_in_Mem &
(~zero_in_Mem)) | Jump_in_Mem[0] | Jump_in_Mem[1];
endmodule

```

② Mem_reg_WB :

PipelineCPU 访存和写回间的寄存器，其中存储了 ALU 数据和存储器相关数据。寄存 Mem 级的输出指令，以及输出数据，传递给写回阶段和寄存器堆。



```

`timescale 1ns / 1ps
module Mem_reg_WB(
    input clk_MemWB,
    input rst_MemWB, //寄存器复位
    input en_MemWB, //寄存器使能
    input[31:0] PC4_in_MemWB, //PC+4 输入
    input[4:0] Rd_addr_MemWB, //写目的地址输入
    input[31:0] ALU_in_MemWB, //ALU 输入
    input[31:0] DMem_data_MemWB, //存储器数据输入
    input[1:0] MemtoReg_in_MemWB, //写回
    input RegWrite_in_MemWB, //寄存器堆读写
    output reg[31:0] PC4_out_MemWB, //PC+4 输出
    output reg[4:0] Rd_addr_out_MemWB, //写目的地址输出
    output reg[31:0] ALU_out_MemWB, //ALU 输出
    output reg[31:0] DMem_data_out_MemWB, //存储器数据输出

```

```

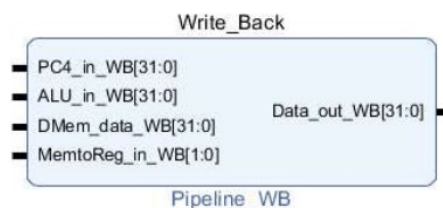
output reg[1:0] MemtoReg_out_MemWB, //写回
output reg RegWrite_out_MemWB //寄存器堆读写;
);
always @(posedge clk_MemWB or posedge rst_MemWB) begin
    if(rst_MemWB == 1'b1) begin
        PC4_out_MemWB <= 32'h0;
        Rd_addr_out_MemWB <= 5'h0;
        ALU_out_MemWB <= 32'h0;
        DMem_data_out_MemWB <= 32'h0;
        MemtoReg_out_MemWB <= 2'h0;
        RegWrite_out_MemWB <= 1'h0;
    end
    else if(en_MemWB == 1'b1) begin
        PC4_out_MemWB <= PC4_in_MemWB;
        Rd_addr_out_MemWB <= Rd_addr_MemWB;
        ALU_out_MemWB <= ALU_in_MemWB;
        DMem_data_out_MemWB <= DMem_data_MemWB;
        MemtoReg_out_MemWB <= MemtoReg_in_MemWB;
        RegWrite_out_MemWB <= RegWrite_in_MemWB;
    end
end
end
endmodule

```

3.3 设计实现写回（WB）模块：

① Pipeline_WB :

PipelineCPU 第五阶段，将结果写回寄存器堆的操作。如果是普通运算指令，该结果值来源于 Ex 阶段计算的结果；如果是 LOAD 指令，该结果来源于 Mem 阶段从存储器读取出来的数据；如果是跳转 (jal) 指令，该结果来源于 PC+4



```

module Pipeline_WB(
    input[31:0] PC4_in_WB, //PC+4 输入
    input[31:0] ALU_in_WB, //ALU 结果输出
    input[31:0] DMem_data_WB, //存储器数据输入
    input[1:0] MemtoReg_in_WB, //写回选择控制
    output [31:0] Data_out_WB //写回数据输出
);

```

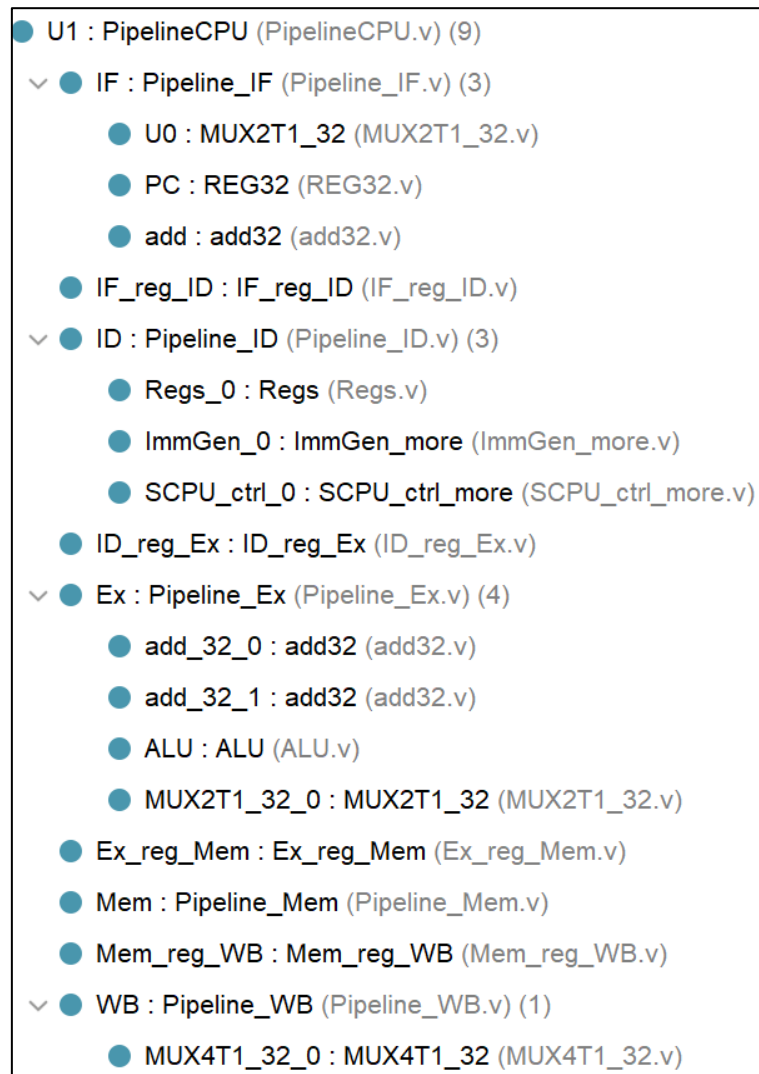
```

MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg_in_WB),
    .I0(ALU_in_WB),
    .I1(DMem_data_WB),
    .I2(PC4_in_WB),
    .I3(PC4_in_WB),
    .o(Data_out_WB)
);
endmodule

```

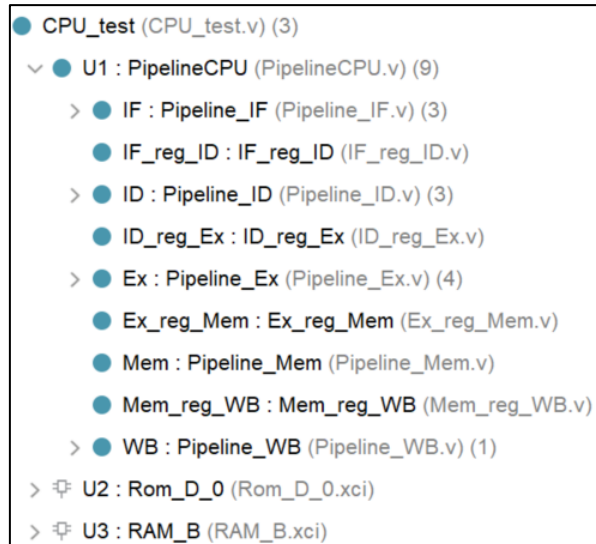
3.2 模块替换、集成流水线 CPU:

利用 Lab5-1 设计的 Pipeline_CPU.v 文件，将提供的流水线各阶段文件（.edf 文件）替换为 Lab5-2~3 实现的模块，以下为替换后 Pipeline_CPU 的工程结构：



3.3.1 搭建仿真平台

同样的，利用 PipelineCPU、RAM、ROM 可以搭建如下图的仿真平台（需要调整 PipelineCPU 模块的输入信号），初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（P_mem.coe），查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。



```
module CPU_test(
    input clk,
    input rst
);
    wire [31:0] Data_in, inst_IF, Addr_out, Data_out, PC_out_IF,
    PC_out_ID, PC_out_EX, Data_out_WB, inst_ID;
    wire MemRW_EX, MemRW_Mem;

    PipelineCPU U1(
        .clk(clk),
        .rst(rst),
        .Data_in(Data_in),
        .inst_IF(inst_IF),
        .PC_out_IF(PC_out_IF),
        .PC_out_ID(PC_out_ID),
        .inst_ID(inst_ID),
        .PC_out_IDEX(PC_out_EX),
        .MemRW_EX(MemRW_EX),
        .MemRW_Mem(MemRW_Mem),
        .Addr_out(Addr_out),
        .Data_out(Data_out),
        .Data_out_WB(Data_out_WB)
    );
```


3.3.2 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。
本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格（见实验数据分析）。

- ROM 中填充指令如下：

P_Mem（无冒险）

```
1 memory_initialization_radix=16;
2 memory_initialization_vector=
3 00100093,
4 00100113,
5 00100193,
6 00100213,
7 00802283,
8 00108333,
9 0020C3B3,
10 40110433,
11 05C02483,
12 00327533,
13 00502223,
14 00532583,
15 0AA3C613,
16 0012D6B3,
17 00147713,
18 0034E7B3,
19 00A50833,
20 0085C8B3,
21 00402903,
22 004629B3,
23 0016DA13,
24 00677AB3,
25 40128B33,
26 00150B93,
27 00986C33,
28 00B9CCB3,
29 0FFA7D13,
30 00390DB3,
31 002A5E33,
32 0AF9EE93,
33 001A0F33,
34 00802F83,
35 F81FF06F,
36 00000033,
37 00000033,
38 00000033;
```

h_Mem（有冒险）

```
1 memory_initialization_radix=16;
2 memory_initialization_vector=
3 00100093,
4 00100113,
5 00100193,
6 00100213,
7 00802283,
8 00128333,
9 0020C3B3,
10 40708433,
11 05C02483,
12 00327533,
13 00502223,
14 00532583,
15 0AA3C613,
16 00818663,
17 00000013,
18 00000033,
19 0012D6B3,
20 00147713,
21 0034E7B3,
22 00A50833,
23 0085C8B3,
24 00402903,
25 004629B3,
26 0016DA13,
27 00677AB3,
28 01071463,
29 00000013,
30 40128B33,
31 00150B93,
32 00986C33,
33 00B9CCB3,
34 0FFA7D13,
35 00390DB3,
36 002A5E33,
37 0AF9EE93,
38 001A0F33,
39 00802F83,
40 F6DFF06F,
41 00000033,
42 00000033,
43 00000033;
```

- RAM 中储存的数据（D_mem.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7DBFDB9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFBFCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7DBFDB9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300
```

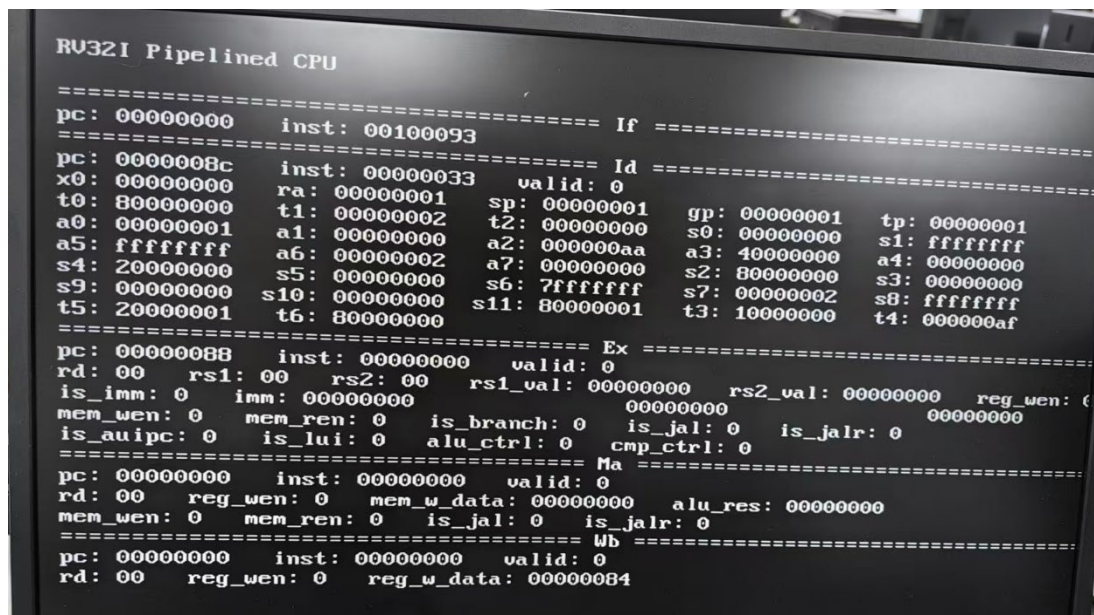
部分测试 VGA 显示图片见实验结果分析

四、实验结果分析

流水线 CPU 实验的物理验证类似单周期 CPU，主要看基本指令能否实现？寄存器数值是否根据对应指令正确修改？以及流水线各阶段能否连续、按序更新 PC 以及执行指令。

1	PC	Machine	Code Basic	Result
2	0x0	0x00100093	addi x1 x0 1	x1 = 0x1
3	0x4	0x00100113	addi x2 x0 1	x2 = 0x1
4	0x8	0x00100193	addi x3 x0 1	x3 = 0x1
5	0xc	0x00100213	addi x4 x0 1	x4 = 0x1
6	0x10	0x00802283	lw x5 8(x0)	x5 = 0x80000000
7	0x14	0x00108333	add x6 x1 x1	x6 = 0x2
8	0x18	0x0020C3B3	xor x7 x1 x2	x7 = 0x0
9	0x1c	0x40110433	sub x8 x2 x1	x8 = 0x0
10	0x20	0x05C02483	lw x9 92(x0)	x9 = 0xFFFFFFFF
11	0x24	0x00327533	and x10 x4 x3	x10 = 0x1
12	0x28	0x00502223	sw x5 4(x0)	mem[1] = 0x80000000
13	0x2c	0x005325B3	slt x11 x6 x5	x11 = 0x0
14	0x30	0x0AA3C613	xori x12 x7 170	x12 = 0xAA
15	0x34	0x0012D6B3	srl x13 x5 x1	x13 = 0x40000000
16	0x38	0x00147713	andi x14 x8 1	x14 = 0x0
17	0x3c	0x0034E7B3	or x15 x9 x3	x15 = 0xFFFFFFFF
18	0x40	0x00A50833	add x16 x10 x10	x16 = 0x2
19	0x44	0x0085C8B3	xor x17 x11 x8	x17 = 0x0
20	0x48	0x00402903	lw x18 4(x0)	x18 = 0x80000000
21	0x4c	0x004629B3	slt x19 x12 x4	x19 = 0x0
22	0x50	0x0016DA13	srli x20 x13 1	x20 = 0x20000000
23	0x54	0x00677AB3	and x21 x14 x6	x21 = 0x0
24	0x58	0x40128B33	sub x22 x5 x1	x22 = 0x7FFFFFFF
25	0x5c	0x00150B93	addi x23 x10 1	x23 = 0x2
26	0x60	0x00986C33	or x24 x16 x9	x24 = 0xFFFFFFFF
27	0x64	0x00B9CCB3	xor x25 x19 x11	x25 = 0x0
28	0x68	0x0FFA7D13	andi x26 x20 25	x26 = 0x0
29	0x6c	0x00390DB3	add x27 x18 x3	x27 = 0x80000001
30	0x70	0x002A5E33	srl x28 x20 x2	x28 = 0x10000000
31	0x74	0x0AF9EE93	ori x29 x19 175	x29 = 0xAF
32	0x78	0x001A0F33	add x30 x20 x1	x30 = 0x20000001
33	0x7c	0x00802F83	lw x31 8(x0)	x31 = 0x80000000
34	0x80	0xF81FF06F	jal x0 -128	go to 0x0
35	0x84	0x00000033	add x0 x0 x0	NOP -- 统一Pipeline各阶段
36	0x88	0x00000033	add x0 x0 x0	
37	0x8c	0x00000033	add x0 x0 x0	

以上表格记录了 P_mem.coe 中的正常程序流（无冒险版本），并记录了运行后各寄存器的数值。



上图是利用 DEMO 物理验证的 VGA 显示，主要检查两个方面：

(1) 寄存器数值：

各寄存器十分符合表格中的程序流结果，对。

(2) 流水线各阶段指令：

上图可以看到 IF、ID、Ex 的 PC 值（Mem、WB 未导出），但是可以简单推出此时 PC-Mem = 0x84，PC-WB = 0x80。从而验证了各阶段的 PC 值都是正确更新的，达到了 PipelineCPU 的实验要求。

0x80	0xF81FF06F	jal x0 -128	go to 0x0
0x84	0x00000033	add x0 x0 x0	NOP -- 统一Pipeline各阶段
0x88	0x00000033	add x0 x0 x0	
0x8c	0x00000033	add x0 x0 x0	

五、讨论与心得

相较于折磨的单周期 CPU 来说，流水线 CPU 也很折磨（特别是还要对官方 pdf 和 PPT 进行纠错。。）。但是，从实现的总体过程来说还是能够让自己比较好地理解 Pipeline 的运行原理和过程。

（Lab5-04 的结果分析会比较详细）

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 12 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: 流水线处理器 — 冒险与 stall

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 马致远

实验地点: 东 4-509 实验日期: 2024 年 12 月 11 日

一、实验目的和要求

- 1.理解流水线 CPU 的基本原理和组织结构
- 2.掌握五级流水线的工作过程和设计方法
- 3.理解流水线 CPU 停机的原理与解决办法
- 4.设计流水线测试程序

二、实验内容和原理

内容:

任务一: 集成设计利用 stall 解决冒险的流水线 CPU

- 设计冒险检测及 stall 消除冒险的流水线 CPU
- 替换 lab05-3 的 CPU 为本实验集成的带 stall 处理的流水线 CPU

任务二: 设计流水线 stall 测试方案并完成测试

原理：

2.1 流水线冒险基本概念

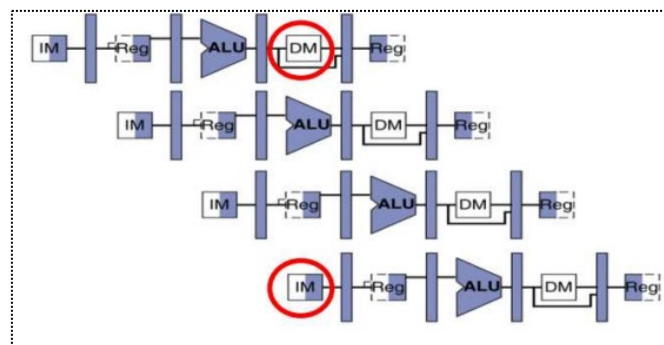
流水线冒险指在下一个时钟周期中下一条指令无法正常执行，引起冒险的原因有多种，主要分为以下三种：

- 结构冒险 (Structural hazard)：硬件不支持多条指令在同一时钟周期执行
- 数据冒险 (Data hazard)：当前指令的执行需要等待前一条指令的数据结果
- 控制冒险 (Control hazard)：指令非顺序执行而导致下一条执行的指令不是真实期望的。

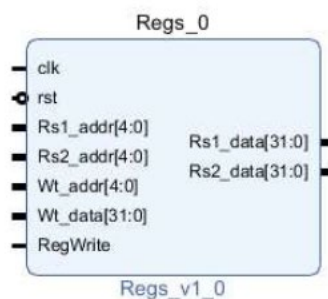
2.2 结构冒险 (Structural hazard)：

也称为硬件资源冲突，指流水线执行期间，两条及以上指令同一时间对同一个硬件资源发起使用的请求。

常见的结构冒险主要发生在 Memory conflicts(指令和数据)、Register conflicts。此次流水线中指令(PC 储存)、数据(Data Memory 储存)，前者(见下图)不会发生。



同时此次流水线中 Register 的读写端口分离，因此在 WB 和 ID 中对 regs 的同时读写并不会冲突。



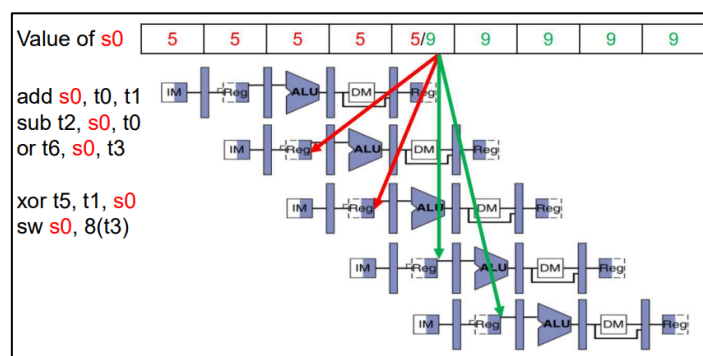
总而言之，本次中断实验不考虑结构冒险的存在。

2.3 数据冒险 (Data hazard):

指后一条指令的执行需要前一条指令的执行结果，而此时前一条指令的结果还未产生，主要有三种情况（指令 i 先于 j）：

- **RAW (Read After Write)**: 当指令 i 写回结果之前，指令 j 已经对此结果发起读操作；
- **WAW (Write After Write)**: 当指令 i 写回结果之前，指令 j 已经将结果写回；
- **WAR (Write After Read)**: 当指令 i 读取数据之前，指令 j 已经将结果写回。

本次实验主要处理 RAW 数据冒险，如下情况：



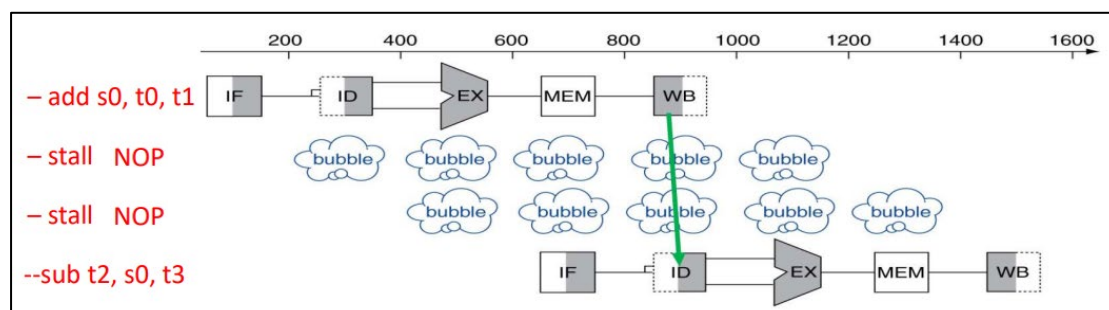
指令 SUB 和 OR 的源操作数都依赖于 ADD 的计算结果，若不采取应对措施，则在 S₀ 结果写回之前，错误的值会被提前读走（xor 时寄存器前写后读）。

2.3.1 数据冒险解决方法:

(1) Solution 1 : Stalling or NOP (• 本实验采用 •)

• 硬件解决方法：流水线阻塞 (stall)，使数据相关的后续指令延迟执行，也称为插入气泡 (bubble)。

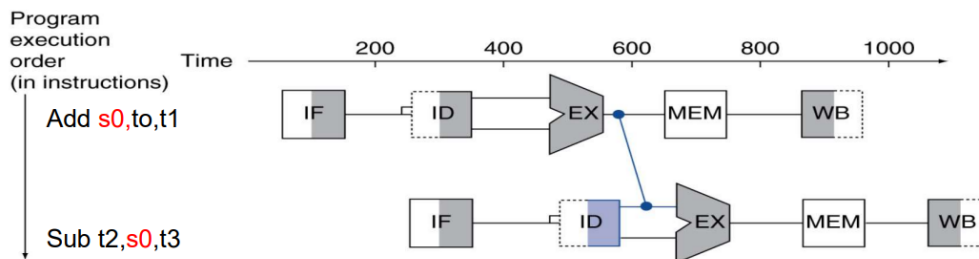
• 软件解决方法：插入空操作 (NOP: addi x0, x0, x0)，使数据相关的后续指令延迟执行。



两种方式虽然都使得流水线能够正常的执行操作，但延迟下一条指令执行的方式使 CPU 性能大打折扣。

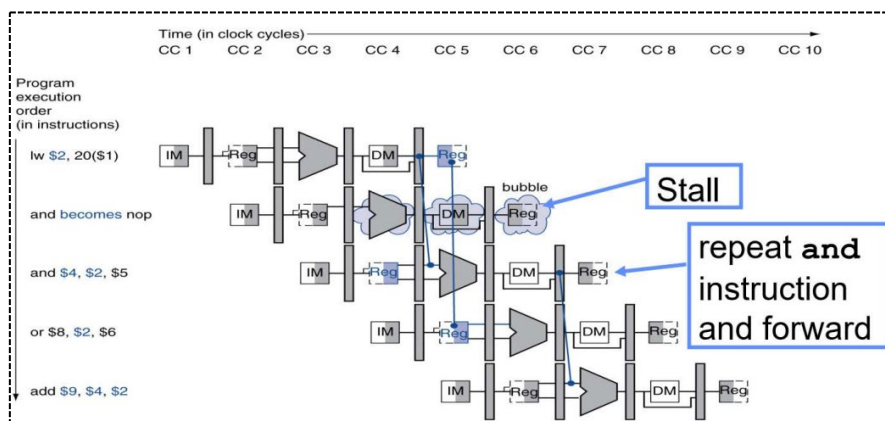
(2) Solution 2 : Forwarding

前递 (forwarding), 又称为旁路 (bypassing) 将数据通路生成的中间数据直接往前传递到 ALU 的输入端, 参与下一条指令的运算。



如上图, 将 ADD 指令执行阶段的输出值 (ALU 结果) 前递到 SUB 指令的执行阶段的输入, 替换 SUB 指令在第二阶段读出的寄存器 S0 的值 (无需等待寄存器的值被写回, 避免 RAW)。

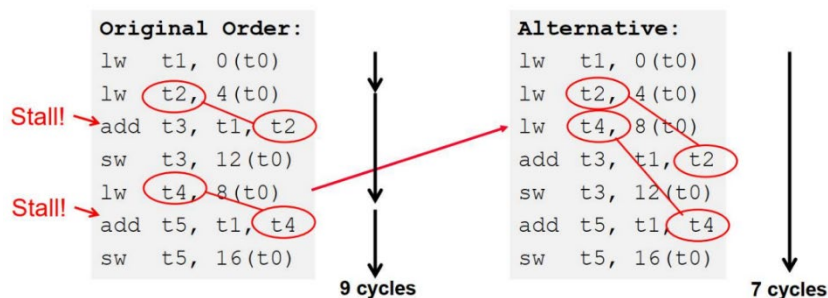
但一次前递只能解决数学运算之间的 Data Hazard, 当一条 load 指令后为一条需要使用其结果的 R 型指令后, 直接前递是无效的 (load 后数据在 Datamemory 中读取, 而非在 ALU 中, 则 Stall 无法避免的)。



(3) Solution 3 : Scheduling

编译器进行指令顺序调整来解决数据冒险。

• eg: RISC-V code for $A[3] = A[0] + A[1]$;
 $A[4] = A[0] + A[2]$



2.3.2 数据冒险检测机制:

(1) Mem Hazard:

(k) th 指令访存 (Ex) 阶段写回寄存器和 (k+1) th 指令译码 (ID) 阶段源寄存器相同, 同时 (k) th 指令数据写回寄存器 (EX/MEM.RegWrite=1)。

- if (EX/MEM.RegWrite and Rs1_used) and (ID.RegisterRs1 $\neq$ 0)
and (EX/MEM.RegisterRd = ID.RegisterRs1)) **Data_stall = 1**
- if (EX/MEM.RegWrite and Rs2_used) and (ID.RegisterRs2 $\neq$ 0)
and (EX/MEM.RegisterRd = ID.RegisterRs2)) **Data_stall = 1**

(2) Ex Hazard :

(k) th 指令执行 (Ex) 阶段写回寄存器和 (k+1) th 指令译码 (ID) 阶段源寄存器相同, 同时 (k) th 指令将有数据写回寄存器 (ID/EX.RegWrite=1)。

- if (ID/EX.RegWrite and Rs1_used) and (ID.RegisterRs1 $\neq$ 0)
and (ID/EX.RegisterRd = ID.RegisterRs1)) **Data_stall = 1**
- if (ID/EX.RegWrite and Rs2_used) and (ID.RegisterRs2 $\neq$ 0)
and (ID/EX.RegisterRd = ID.RegisterRs2)) **Data_stall = 1**

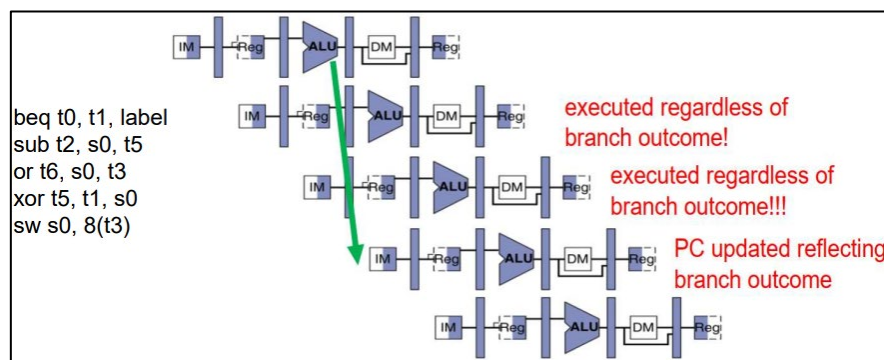
检测到 Data Hazard , 拉高 Data_stall 标志信号, 此时采取停机操作, 在译码和执行之间插入 NOP 指令:

```
if (Data_stall == 1'b1) begin
    en_IF = 0;
    en_IFID = 0;
    NOP_IDEX = 1;
end
else begin
    en_IF = 1;
    en_IFID = 1;
    NOP_IDEX = 0;
end
```

2.4 控制冒险

当 PC 值不按顺序执行时,流水线中指令的正常执行会被阻塞,常见的如 beq、bne、jal 等跳转指令。

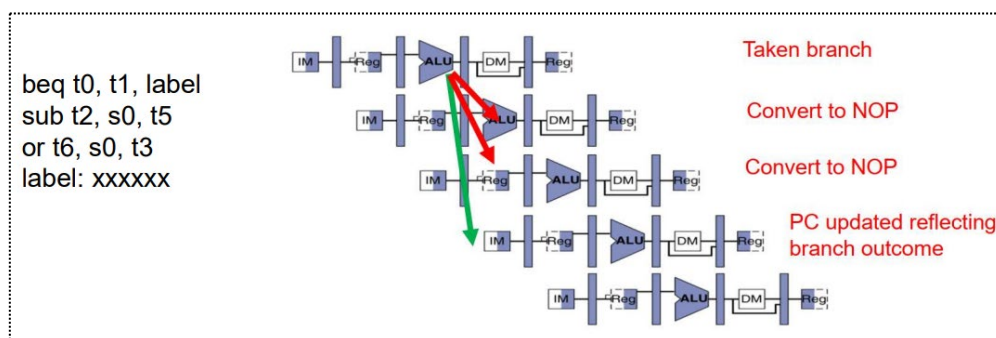
如图, beq 指令 (原计划在 ALU 计算结束后跳转到 xor 指令) 执行之后, 下一条指令应该等待其执行完成才知道目标地址, 此时若不采取措施, pc 会顺序的开始 sub 操作, 显然不是期望的结果:



2.4.1 控制冒险解决方法

(1) Solution 1 : Stalling or NOP (• 本实验采用 •)

采用类似 Data Hazard 的解决, 采用插入 NOP 的方式解决 (同样, 容易执行, 但是对 Pipeline 性能影响较大)。



(2) Solution 2 : Branch Prediction

分支预测 (Branch Prediction): 分为静态预测 (简单预测分支指令的条件总是满足或不满足) 和动态预测 (根据实际执行情况动态调整预测位), 预测准确则无时间损失, 若预测不准确则对分支后不该执行的指令进行冲刷。

从而最大限度减少控制冒险对性能的影响。

2.4.2 控制冒险检测机制

本实验会改变 PC 的指令为 beq;bne;jal; 因此只需检测这三条指令（采用拓展后的 SCPU_ctrl，因此 Jump 考虑了 jalr）的特征信号即可：

```
if ((Branch_ID || BranchN_ID || Jump_ID != 2'b00) ||  
    // 译码阶段  
    (Branch_out_IDEX || BranchN_out_IDEX || Jump_out_IDEX != 2'b00) ||  
    // 执行阶段  
    (Branch_out_EXMem || BranchN_out_EXMem || Jump_out_EXMem != 2'b00))  
    // 访存阶段  
begin  
    Control_stall = 1'b1;  
end  
else  
    Control_stall = 1'b0;
```

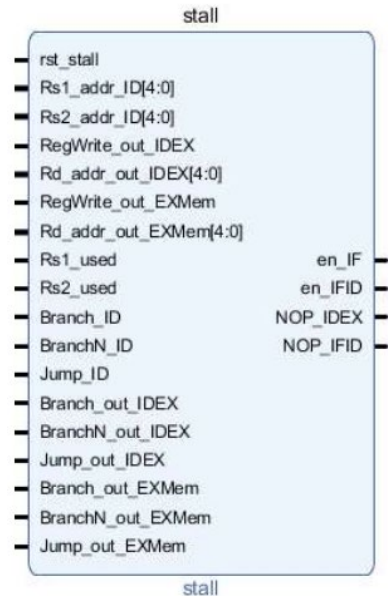
检测到 Control Hazard，拉高 Control_stall 标志信号，此时采取停机操作，在取指和译码之间插入 NOP 指令：

```
if (Control_stall == 1'b1)  
    NOP_IFID = 1;  
else  
    NOP_IFID = 0;
```

三、实验过程和数据记录

3.1 集成设计利用 stall 解决冒险的流水线 CPU:

(1) Stall —— 冒险检测及处理单元:



```
`timescale 1ns / 1ps
module stall(
    input rst_stall, //复位
    input RegWrite_out_IDEX, //执行阶段寄存器写控制
    input [4:0]Rd_addr_out_IDEX, //执行阶段寄存器写地址
    input RegWrite_out_EXMem, //访存阶段寄存器写控制
    input [4:0]Rd_addr_out_EXMem, //访存阶段寄存器写地址
    input [4:0]Rs1_addr_ID, //译码阶段寄存器读地址 1
    input [4:0]Rs2_addr_ID, //译码阶段寄存器读地址 2
    input Rs1_used, //Rs1 被使用
    input Rs2_used, //Rs2 被使用
    input Branch_ID, //译码阶段 beq
    input BranchN_ID, //译码阶段 bne
    input [1:0]Jump_ID, //译码阶段 jal
    input Branch_out_IDEX, //执行阶段 beq
    input BranchN_out_IDEX, //执行阶段 bne
    input [1:0]Jump_out_IDEX, //执行阶段 jal
    input Branch_out_EXMem, //访存阶段 beq
    input BranchN_out_EXMem, //访存阶段 bne
    input [1:0]Jump_out_EXMem,
    output reg en_IF, //流水线寄存器的使能及 NOP 信号
    output reg en_IFID,
    output reg NOP_IFID,
    output reg NOP_IDEX);
```

```

reg Data_stall, Control_stall;

always@(*) begin
    if(rst_stall) begin
        en_IF = 1;
        en_IFID = 1;
        NOP_IFID = 0;
        NOP_IDEX = 0;
    End

    if ((Rs1_used && (Rs1_addr_ID == Rd_addr_out_IDEX) &&
(Rs1_addr_ID != 5'h0) && RegWrite_out_IDEX) ||
        (Rs2_used && (Rs2_addr_ID == Rd_addr_out_IDEX) &&
(Rs2_addr_ID != 5'h0) && RegWrite_out_IDEX) ||
        (Rs1_used && (Rs1_addr_ID == Rd_addr_out_EXMem) &&
(Rs1_addr_ID != 5'h0) && RegWrite_out_EXMem) ||
        (Rs2_used && (Rs2_addr_ID == Rd_addr_out_EXMem) &&
(Rs2_addr_ID != 5'h0) && RegWrite_out_EXMem))
        begin
            Data_stall = 1'b1;
        end
    else
        Data_stall = 1'b0;
        if ((Branch_ID || BranchN_ID || Jump_ID != 2'b00) ||
            (Branch_out_IDEX || BranchN_out_IDEX || Jump_out_IDEX != 2'b00) ||
            (Branch_out_EXMem || BranchN_out_EXMem || Jump_out_EXMem != 2'b00))
            Control_stall = 1'b1;
        else
            Control_stall = 1'b0;
        if (Data_stall == 1'b1) begin
            en_IF = 0;
            en_IFID = 0;
            NOP_IDEX = 1;
        end
        else begin
            en_IF = 1;
            en_IFID = 1;
            NOP_IDEX = 0;
        end
        if (Control_stall == 1'b1)
            NOP_IFID = 1;
        else
            NOP_IFID = 0;
    end
end

```

根据 Stall 中所要求的内容，涉及了寄存器堆的读写地址信号，以及 Rs1、Rs2 的使用情况，因此译码阶段 Pipeline_ID 需要微调。

同时检测到 Hazard 后，我们得到了 en_IF、en_IFID、NOP_IDEX、NOP_IFID 等信号，因此需要对流水线各级寄存器（特别是 IF_reg_ID、ID_reg_EX 两个需要信号中断的寄存器）进行修改。

（2） IF_reg_ID：

• 接口修改：

```
module IF_reg_ID(  
    input clk_IFID,  
    input rst_IFID,  
    input en_IFID,  
    input NOP_IFID, // 插入 NOP 使能  
    input [31:0]PC_in_IFID,  
    input [31:0]inst_in_IFID,  
    output reg [31:0]PC_out_IFID,  
    output reg [31:0]inst_out_IFID,  
    output reg valid_IFID  
);
```

• 逻辑修改：

```
always @(posedge clk_IFID or posedge rst_IFID)  
    if (rst_IFID == 1'b1) begin  
        PC_out_IFID <= 32'h0;  
        inst_out_IFID <= 32'h0;  
        valid_IFID <= 1'b1;  
    end  
    else if(NOP_IFID) begin  
        PC_out_IFID <= 32'h0;  
        inst_out_IFID <= 32'h00000013; // 插入空指令 add x0, x0, x0  
        valid_IFID <= 1'b0;  
    end  
    else if (en_IFID == 1'b1) begin  
        PC_out_IFID <= PC_in_IFID;  
        inst_out_IFID <= inst_in_IFID;  
        valid_IFID <= 1'b1;  
    end  
end
```

(3) Pipeline_ID :

从 Stall 模块中知道，我们需要译码时增加以下四个信号：

```
input [4:0]Rs1_addr_ID, //译码阶段寄存器读地址 1
input [4:0]Rs2_addr_ID, //译码阶段寄存器读地址 2
input Rs1_used, //Rs1 被使用
input Rs2_used, //Rs2 被使用
```

由于这些内容可以直接从指令中读出，因此不用修改 SCPU unit:

```
assign Rs1_addr_ID = Inst_in_ID[19:15];
assign Rs2_addr_ID = Inst_in_ID[24:20];
assign Rs1_used = (Inst_in_ID[6:0] == 7'b0110011) || // R-type 指令
                  (Inst_in_ID[6:0] == 7'b0000011) || // I-type 指令
                  (Inst_in_ID[6:0] == 7'b1100011); // B-type 指令 (
assign Rs2_used = (Inst_in_ID[6:0] == 7'b0110011) || // R-type 指令
                  (Inst_in_ID[6:0] == 7'b0100011) || // S-type 指令
                  (Inst_in_ID[6:0] == 7'b1100011); // B-type 指令
```

(4) ID_reg_Ex :

此寄存器也需要简单的改动，对 NOP_IDEX 信号做出反应，同时输出 valid 信号指出此时是否寄存器有效：

```
module ID_reg_Ex(
    input clk_IDEX, //寄存器时钟
    input rst_IDEX, //寄存器复位
    input en_IDEX, //寄存器使能

    input NOP_IDEX, //插入 NOP 使能
    input valid_in_IDEX, //有效
    input[31:0] Inst_in_IDEX,

    input[31:0] PC_in_IDEX, //PC 输入
    input[4:0] Rd_addr_IDEX, //写目的地址输入
    input[31:0] Rs1_in_IDEX, //操作数 1 输入
    input[31:0] Rs2_in_IDEX, //操作数 2 输入
    input[31:0] Imm_in_IDEX, //立即数输入
    input ALUSrc_B_in_IDEX, //ALU B 输入选择
    input[3:0] ALU_control_in_IDEX, //ALU 选择控制
    input Branch_in_IDEX, //Beq
    input BranchN_in_IDEX, //Bne
    input MemRW_in_IDEX, //存储器读写
    input[1:0] Jump_in_IDEX, //Jal
    input[1:0] MemtoReg_in_IDEX, //写回选择
    input RegWrite_in_IDEX, //寄存器堆读写
```

```

output reg[31:0] PC_out_IDEX, //PC 输出
output reg[31:0] Inst_out_IDEX, // inst 输出
output reg valid_out_IDEX,

output reg[4:0] Rd_addr_out_IDEX, //目的地址输出
output reg[31:0] Rs1_out_IDEX, //操作数 1 输出
output reg[31:0] Rs2_out_IDEX, //操作数 2 输出
output reg[31:0] Imm_out_IDEX , //立即数输出
output reg ALUSrc_B_out_IDEX, //ALU B 选择
output reg[3:0] ALU_control_out_IDEX, //ALU 控制
output reg Branch_out_IDEX, //Beq
output reg BranchN_out_IDEX, //Bne
output reg MemRW_out_IDEX, //存储器读写
output reg[1:0] Jump_out_IDEX, //Jal
output reg[1:0] MemtoReg_out_IDEX, //写回
output reg RegWrite_out_IDEX //寄存器堆读写
);

```

• 逻辑修改:

```

always @(posedge clk_IDEX or posedge rst_IDEX) begin
    if(rst_IDEX == 1'b1)begin
        PC_out_IDEX <= 32'h0;
        Inst_out_IDEX <= 32'h0;
        Rd_addr_out_IDEX <= 5'h0;
        Rs1_out_IDEX <= 32'h0;
        Rs2_out_IDEX <= 32'h0;
        Imm_out_IDEX <= 32'h0;
        ALUSrc_B_out_IDEX <= 1'h0;
        ALU_control_out_IDEX <= 4'h0;
        Branch_out_IDEX <= 1'h0;
        BranchN_out_IDEX <= 1'h0;
        MemRW_out_IDEX <= 1'h0;
        Jump_out_IDEX <= 2'h0;
        MemtoReg_out_IDEX <= 2'h0;
        RegWrite_out_IDEX <= 1'h0;
        valid_out_IDEX <= 1'b1;
    end
    else if(NOP_IDEX) begin
        // else if(NOP_IDEX || valid_in_IDEX == 1'b0) begin
        PC_out_IDEX <= 32'h0;
        Inst_out_IDEX <= 32'h00000013;
        Rd_addr_out_IDEX <= 5'h0;
        Rs1_out_IDEX <= 32'h0;
        Rs2_out_IDEX <= 32'h0;
        Imm_out_IDEX <= 32'h0;
    end
end

```

```

        ALUSrc_B_out_IDEX <= 1'h0;
        ALU_control_out_IDEX <= 4'h0;
        Branch_out_IDEX <= 1'h0;
        BranchN_out_IDEX <= 1'h0;
        MemRW_out_IDEX <= 1'h0;
        Jump_out_IDEX <= 2'h0;
        MemtoReg_out_IDEX <= 2'h0;
        RegWrite_out_IDEX <= 1'h0;

        valid_out_IDEX <= 1'b0;
    end
    else if(en_IDEX == 1'b1) begin
        PC_out_IDEX <= PC_in_IDEX;
        Inst_out_IDEX <= Inst_in_IDEX;
        Rd_addr_out_IDEX <= Rd_addr_IDEX;
        Rs1_out_IDEX <= Rs1_in_IDEX;
        Rs2_out_IDEX <= Rs2_in_IDEX;
        Imm_out_IDEX <= Imm_in_IDEX;
        ALUSrc_B_out_IDEX <= ALUSrc_B_in_IDEX;
        ALU_control_out_IDEX <= ALU_control_in_IDEX;
        Branch_out_IDEX <= Branch_in_IDEX;
        BranchN_out_IDEX <= BranchN_in_IDEX;
        MemRW_out_IDEX <= MemRW_in_IDEX;
        Jump_out_IDEX <= Jump_in_IDEX;
        MemtoReg_out_IDEX <= MemtoReg_in_IDEX;
        RegWrite_out_IDEX <= RegWrite_in_IDEX;
        valid_out_IDEX <= 1'b1;
    end
end
end

```

(5) Ex_reg_Mem :

此后的两个寄存器不需要对 NOP 进行相应,但是可以根据 valid(可以说 valid 的作用约等于之前 NOP 的作用)来判断目前是否需要进行操作:

```

`timescale 1ns / 1ps

module Ex_reg_Mem(
    input clk_EXMem,
    input rst_EXMem,
    input en_EXMem,

    input [31:0]PC_imm_EXMem,
    input valid_in_EXMem,

```

```

input [31:0]Inst_in_EXMem,

input [31:0]PC_in_EXMem,
input [31:0]PC4_in_EXMem,
input [4:0]Rd_addr_EXMem,
input zero_in_EXMem,
input [31:0]ALU_in_EXMem,
input [31:0]Rs2_in_EXMem,
input Branch_in_EXMem,
input BranchN_in_EXMem,
input MemRW_in_EXMem,
input [1:0]Jump_in_EXMem,
input [1:0]MemtoReg_in_EXMem,
input RegWrite_in_EXMem,

output reg[31:0]PC_imm_out_EXMem,
output reg valid_out_EXMem,
output reg[31:0]Inst_out_EXMem,

output reg[31:0]PC_out_EXMem,
output reg[31:0]PC4_out_EXMem,
output reg[4:0]Rd_addr_out_EXMem,
output reg zero_out_EXMem,
output reg[31:0]ALU_out_EXMem,
output reg[31:0]Rs2_out_EXMem,
output reg Branch_out_EXMem,
output reg BranchN_out_EXMem,
output reg MemRW_out_EXMem,
output reg[1:0]Jump_out_EXMem,
output reg[1:0]MemtoReg_out_EXMem,
output reg RegWrite_out_EXMem
);

always @(posedge clk_EXMem or posedge rst_EXMem) begin
    if(rst_EXMem == 1'b1) begin
        PC_imm_out_EXMem <= 32'h0;
        Inst_out_EXMem <= 32'h0;
        PC_out_EXMem <= 32'h0;
        PC4_out_EXMem <= 32'h0;
        Rd_addr_out_EXMem <= 5'h0;
        zero_out_EXMem <= 1'h0;
        ALU_out_EXMem <= 32'h0;
        Rs2_out_EXMem <= 32'h0;
        Branch_out_EXMem <= 1'h0;
    end
end

```

```

        BranchN_out_EXMem <= 1'h0;
        MemRW_out_EXMem <= 1'h0;
        Jump_out_EXMem <= 2'h0;
        MemtoReg_out_EXMem <= 2'h0;
        RegWrite_out_EXMem <= 1'h0;
        valid_out_EXMem <= 1'b1;
    end
else if(valid_in_EXMem == 1'b0) begin
    PC_imm_out_EXMem <= 32'h0;
    Inst_out_EXMem <= 32'h00000013;
    PC_out_EXMem <= 32'h0;
    PC4_out_EXMem <= 32'h0;
    Rd_addr_out_EXMem <= 5'h0;
    zero_out_EXMem <= 1'h0;
    ALU_out_EXMem <= 32'h0;
    Rs2_out_EXMem <= 32'h0;
    Branch_out_EXMem <= 1'h0;
    BranchN_out_EXMem <= 1'h0;
    MemRW_out_EXMem <= 1'h0;
    Jump_out_EXMem <= 2'h0;
    MemtoReg_out_EXMem <= 2'h0;
    RegWrite_out_EXMem <= 1'h0;
    valid_out_EXMem <= 1'b0;
end
else if(en_EXMem) begin
    PC_imm_out_EXMem <= PC_imm_EXMem;
    Inst_out_EXMem <= Inst_in_EXMem;
    PC_out_EXMem <= PC_in_EXMem;
    PC4_out_EXMem <= PC4_in_EXMem;
    Rd_addr_out_EXMem <= Rd_addr_EXMem;
    zero_out_EXMem <= zero_in_EXMem;
    ALU_out_EXMem <= ALU_in_EXMem;
    Rs2_out_EXMem <= Rs2_in_EXMem;
    Branch_out_EXMem <= Branch_in_EXMem;
    BranchN_out_EXMem <= BranchN_in_EXMem;
    MemRW_out_EXMem <= MemRW_in_EXMem;
    Jump_out_EXMem <= Jump_in_EXMem;
    MemtoReg_out_EXMem <= MemtoReg_in_EXMem;
    RegWrite_out_EXMem <= RegWrite_in_EXMem;
    valid_out_EXMem <= 1'b1;
end
end
endmodule

```

(6) Mem_reg_WB :

```
`timescale 1ns / 1ps
module Mem_reg_WB(
    input clk_MemWB, //寄存器时
    input rst_MemWB, //寄存器复位
    input en_MemWB, //寄存器使能
    input[31:0] PC_in_MemWB,
    input[31:0] Inst_in_MemWB,
    inout valid_in_MemWB,

    input[31:0] PC4_in_MemWB, //PC+4 输入
    input[4:0] Rd_addr_MemWB, //写目的地址输入
    input[31:0] ALU_in_MemWB, //ALU 输入
    input[31:0] DMem_data_MemWB, //存储器数据输入
    input[1:0] MemtoReg_in_MemWB, //写回
    input RegWrite_in_MemWB, //寄存器堆读写

    output reg[31:0] PC_out_MemWB,
    output reg[31:0] Inst_out_MemWB,
    output reg valid_out_MemWB,

    output reg[31:0] PC4_out_MemWB, //PC+4 输出
    output reg[4:0] Rd_addr_out_MemWB, //写目的地址输出
    output reg[31:0] ALU_out_MemWB, //ALU 输出
    output reg[31:0] DMem_data_out_MemWB, //存储器数据输出
    output reg[1:0] MemtoReg_out_MemWB, //写回
    output reg RegWrite_out_MemWB //寄存器堆读写;
);
always @(posedge clk_MemWB or posedge rst_MemWB) begin
    if(rst_MemWB == 1'b1) begin
        PC_out_MemWB <= 32'h0;
        Inst_out_MemWB <= 32'h0;
        PC4_out_MemWB <= 32'h0;
        Rd_addr_out_MemWB <= 5'h0;
        ALU_out_MemWB <= 32'h0;
        DMem_data_out_MemWB <= 32'h0;
        MemtoReg_out_MemWB <= 2'h0;
        RegWrite_out_MemWB <= 1'h0;
        valid_out_MemWB <= 1'b1;
    end
    else if(valid_in_MemWB == 1'b0) begin
        PC_out_MemWB <= 32'h0;
        Inst_out_MemWB <= 32'h00000013;
        PC4_out_MemWB <= 32'h0;
```

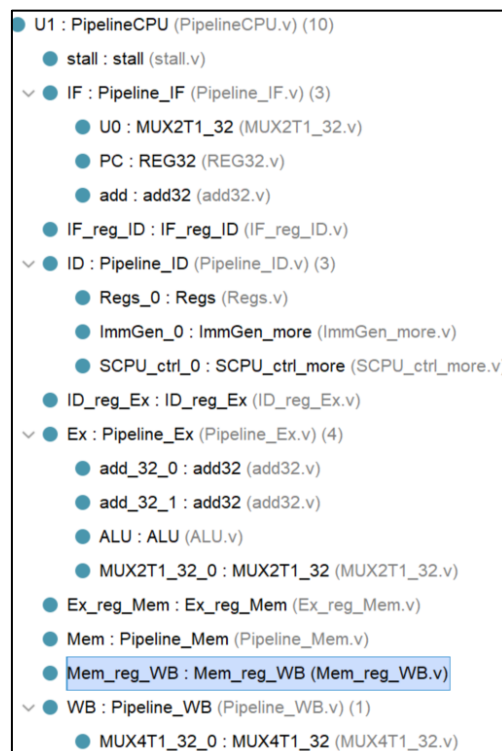
```

        Rd_addr_out_MemWB <= 5'h0;
        ALU_out_MemWB <= 32'h0;
        DMem_data_out_MemWB <= 32'h0;
        MemtoReg_out_MemWB <= 2'h0;
        RegWrite_out_MemWB <= 1'h0;
        valid_out_MemWB <= 1'b0;
    end
    else if(en_MemWB == 1'b1) begin
        PC_out_MemWB <= PC_in_MemWB;
        Inst_out_MemWB <= Inst_in_MemWB;
        PC4_out_MemWB <= PC4_in_MemWB;
        Rd_addr_out_MemWB <= Rd_addr_MemWB;
        ALU_out_MemWB <= ALU_in_MemWB;
        DMem_data_out_MemWB <= DMem_data_MemWB;
        MemtoReg_out_MemWB <= MemtoReg_in_MemWB;
        RegWrite_out_MemWB <= RegWrite_in_MemWB;
        valid_out_MemWB <= 1'b1;
    end
end
endmodule

```

3.2 集成 CPU

通过 3.1 我们通过逻辑代码实现了 Pipeline 对各种冒险的检测以及随后 stall 的处理，首先对各个模块进行集成组合成 PipelineCPU：



【具体代码如下】

```
`timescale 1ns / 1ps

module PipelineCPU(
    input clk,           //时钟
    input rst,           //复位
    input[31:0] Data_in, //存储器数据输入
    input[31:0] inst_IF, //取指阶段指令
    output [31:0] PC_out_IF, //取指阶段 PC 输出
    output [31:0] PC_out_ID, //译码阶段 PC 输出
    output [31:0] inst_ID, //译码阶段指令
    output [31:0] PC_out_IDEX, //执行阶段 PC 输出
    output MemRW_EX, //执行阶段存储器读写
    output MemRW_Mem, //访存阶段存储器读写
    output [31:0] Addr_out, //地址输出
    output [31:0] Data_out, //CPU 数据输出
    output [31:0] Data_out_WB, //写回数据输出
    output [1023:0] all_regs_data //所有寄存器数据输出
);

    wire [31:0] Rs1_out_ID , Rs2_out_ID, Imm_out_ID;
    wire [4:0] Rd_addr_out_ID;
    wire [3:0] ALU_control_ID;
    wire [1:0] MemtoReg_ID, Jump_ID;
    wire ALUSrc_B_ID , Branch_ID , BranchN_ID , MemRW_ID ,
RegWrite_out_ID;

    wire [31:0] PC_out_EX , Rs1_out_IDEX , Rs2_out_IDEX, Imm_out_IDEX;
    wire [4:0] Rd_addr_out_IDEX;
    wire [3:0] ALU_control_IDEX;
    wire [1:0] MemtoReg_IDEX, Jump_IDEX;
    wire ALUSrc_B_IDEX , Branch_IDEX , BranchN_IDEX , MemRW_IDEX ,
RegWrite_IDEX;

    wire [31:0] PC4_out_EX, ALU_out_EX, Rs2_out_EX;
    wire zero_out_EX;

    wire [31:0] PC_out_EXMem , PC4_out_EXMem;
    wire [4:0] Rd_addr_out_EXMem;
    wire [1:0] MemtoReg_out_EXMem, Jump_out_EXMem;
    wire zero_out_EXMem, Branch_out_EXMem, BranchN_out_EXMem,
RegWrite_out_EXMem;

    wire PCSrc; // memory stage
```

```

wire [31:0] PC4_out_MemWB , ALU_out_MemWB, DMem_data_out_MemWB;
wire [4:0] Rd_addr_out_MemWB;
wire [1:0] MemtoReg_out_MemWB;
wire RegWrite_out_MemWB;

wire en_IF , en_IFID , NOP_IFID , NOP_IDEX;

wire[31:0] PC_imm_out_EXMem, Inst_out_EXMem;
wire valid_out_EXMem; // Part 'Ex_reg_Mem'

wire [4:0] Rs1_addr_ID, Rs2_addr_ID;
wire Rs1_used , Rs2_used;

stall stall(
    .rst_stall(rst),
    .Rs1_addr_ID(Rs1_addr_ID),
    .Rs2_addr_ID(Rs2_addr_ID),
    .RegWrite_out_IDEX(RegWrite_IDEX),
    .RegWrite_out_EXMem(RegWrite_out_EXMem),//???
    .Rd_addr_out_IDEX(Rd_addr_out_IDEX),
    .Rd_addr_out_EXMem(Rd_addr_out_EXMem),//???
    .Rs1_used(Rs1_used),
    .Rs2_used(Rs2_used),
    .Branch_ID(Branch_ID),
    .BranchN_ID(BranchN_ID),
    .Jump_ID(Jump_ID),
    .Branch_out_IDEX(Branch_IDEX),
    .BranchN_out_IDEX(BranchN_IDEX),
    .Jump_out_IDEX(Jump_IDEX),
    .Branch_out_EXMem(Branch_out_EXMem),
    .BranchN_out_EXMem(BranchN_out_EXMem),
    .Jump_out_EXMem(Jump_out_EXMem)
    .en_IF(en_IF),
    .en_IFID(en_IFID),
    .NOP_IDEX(NOP_IDEX),
    .NOP_IFID(NOP_IFID)

);

Pipeline_IF IF(
    .clk_IF(clk),
    .rst_IF(rst),
    .en_IF(en_IF),
    .PC_in_IF(PC_imm_out_EXMem),

```

```

        .PCSrc(PCSrc),
        .PC_out_IF(PC_out_IF)
    );

    wire valid_IFID;
    IF_reg_ID IF_reg_ID(
        .clk_IFID(clk),
        .rst_IFID(rst),
        .en_IFID(en_IFID),
        .NOP_IFID(NOP_IFID),
        .PC_in_IFID(PC_out_IF),
        .inst_in_IFID(inst_IF),
        .PC_out_IFID(PC_out_ID),
        .inst_out_IFID(inst_ID),
        .valid_IFID(valid_IFID)
    );

    Pipeline_ID ID(
        .clk_ID(clk),
        .rst_ID(rst),
        .RegWrite_in_ID(RegWrite_out_MemWB),
        .Rd_addr_ID(Rd_addr_out_MemWB),
        .Wt_data_ID(Data_out_WB),
        .Inst_in_ID(inst_ID),
        .Rd_addr_out_ID(Rd_addr_out_ID),
        .Rs1_out_ID(Rs1_out_ID),
        .Rs2_out_ID(Rs2_out_ID),
        .Rs1_addr_ID(Rs1_addr_ID),
        .Rs2_addr_ID(Rs2_addr_ID),
        .Rs1_used(Rs1_used),
        .Rs2_used(Rs2_used),
        .Imm_out_ID(Imm_out_ID),
        .ALUSrc_B_ID(ALUSrc_B_ID),
        .ALU_control_ID(ALU_control_ID),
        .Branch_ID(Branch_ID),
        .BranchN_ID(BranchN_ID),
        .MemRW_ID(MemRW_ID),
        .Jump_ID(Jump_ID),
        .MemtoReg_ID(MemtoReg_ID),
        .RegWrite_out_ID(RegWrite_out_ID),
        .all_regs_data(all_regs_data)
    );

    wire[31:0] Inst_out_IDEX;
    wire valid_out_IDEX;

```

```

ID_reg_Ex ID_reg_Ex(
    .clk_IDEX(clk),
    .rst_IDEX(rst),
    .en_IDEX(1'b1),
    .NOP_IDEX(NOP_IDEX),
    .valid_in_IDEX(valid_IFID),
    .PC_in_IDEX(PC_out_ID),
    .Inst_in_IDEX(inst_ID),
    .Rd_addr_IDEX(Rd_addr_out_ID),
    .Rs1_in_IDEX(Rs1_out_ID),
    .Rs2_in_IDEX(Rs2_out_ID),
    .Imm_in_IDEX(Imm_out_ID),
    .ALUSrc_B_in_IDEX(ALUSrc_B_ID),
    .ALU_control_in_IDEX(ALU_control_ID),
    .Branch_in_IDEX(Branch_ID),
    .BranchN_in_IDEX(BranchN_ID),
    .MemRW_in_IDEX(MemRW_ID),
    .Jump_in_IDEX(Jump_ID),
    .MemtoReg_in_IDEX(MemtoReg_ID),
    .RegWrite_in_IDEX(RegWrite_out_ID),
    .PC_out_IDEX(PC_out_IDEX),
    .Inst_out_IDEX(Inst_out_IDEX),
    .Rd_addr_out_IDEX(Rd_addr_out_IDEX),
    .Rs1_out_IDEX(Rs1_out_IDEX),
    .Rs2_out_IDEX(Rs2_out_IDEX),
    .Imm_out_IDEX(Imm_out_IDEX),
    .ALUSrc_B_out_IDEX(ALUSrc_B_IDEX),
    .ALU_control_out_IDEX(ALU_control_IDEX),
    .Branch_out_IDEX(Branch_IDEX),
    .BranchN_out_IDEX(BranchN_IDEX),
    .MemRW_out_IDEX(MemRW_EX),
    .Jump_out_IDEX(Jump_IDEX),
    .MemtoReg_out_IDEX(MemtoReg_IDEX),
    .RegWrite_out_IDEX(RegWrite_IDEX),
    .valid_out_IDEX(valid_out_IDEX)
);

Pipeline_Ex Ex(
    .PC_in_EX(PC_out_IDEX),
    .Rs1_in_EX(Rs1_out_IDEX),
    .Rs2_in_EX(Rs2_out_IDEX),
    .Imm_in_EX(Imm_out_IDEX),
    .ALUSrc_B_in_EX(ALUSrc_B_IDEX),
    .ALU_control_in_EX(ALU_control_IDEX),

```

```

        .PC_out_EX(PC_out_EX),
        .PC4_out_EX(PC4_out_EX),
        .zero_out_EX(zero_out_EX),
        .ALU_out_EX(ALU_out_EX),
        .Rs2_out_EX(Rs2_out_EX)
    );
    Ex_reg_Mem Ex_reg_Mem(
        .clk_EXMem(clk),
        .rst_EXMem(rst),
        .en_EXMem(1'b1),
        .PC_imm_EXMem(PC_out_EX),
        .PC_in_EXMem(PC_out_IDEX),
        .PC4_in_EXMem(PC4_out_EX),
        .valid_in_EXMem(valid_out_IDEX),
        .Inst_in_EXMem(Inst_out_IDEX),
        .Rd_addr_EXMem(Rd_addr_out_IDEX),
        .zero_in_EXMem(zero_out_EX),
        .ALU_in_EXMem(ALU_out_EX),
        .Rs2_in_EXMem(Rs2_out_EX),
        .Branch_in_EXMem(Branch_IDEX),
        .BranchN_in_EXMem(BranchN_IDEX),
        .MemRW_in_EXMem(MemRW_EX),
        .Jump_in_EXMem(Jump_IDEX), // 'jump' 好一个错误
        .MemtoReg_in_EXMem(MemtoReg_IDEX),
        .RegWrite_in_EXMem(RegWrite_IDEX),
        .PC_imm_out_EXMem(PC_imm_out_EXMem),
        .Inst_out_EXMem(Inst_out_EXMem),
        .valid_out_EXMem(valid_out_EXMem),
        .PC_out_EXMem(PC_out_EXMem),
        .PC4_out_EXMem(PC4_out_EXMem),
        .Rd_addr_out_EXMem(Rd_addr_out_EXMem),
        .zero_out_EXMem(zero_out_EXMem),
        .ALU_out_EXMem(Addr_out),
        .Rs2_out_EXMem(Data_out),
        .Branch_out_EXMem(Branch_out_EXMem),
        .BranchN_out_EXMem(BranchN_out_EXMem),
        .MemRW_out_EXMem(MemRW_Mem),
        .Jump_out_EXMem(Jump_out_EXMem),
        .MemtoReg_out_EXMem(MemtoReg_out_EXMem),
        .RegWrite_out_EXMem(RegWrite_out_EXMem)
    );
    Pipeline_Mem Mem(
        .zero_in_Mem(zero_out_EXMem),
        .Branch_in_Mem(Branch_out_EXMem),

```

```

        .BranchN_in_Mem(BranchN_out_EXMem),
        .Jump_in_Mem(Jump_out_EXMem),
        .PCSrc(PCSrc)
    );

    wire [31:0] PC_out_MemWB , Inst_out_MemWB;
    wire valid_out_MemWB;

    Mem_reg_WB Mem_reg_WB(
        .clk_MemWB(clk),
        .rst_MemWB(rst),
        .en_MemWB(1'b1),
        .PC_in_MemWB(PC_out_EXMem),
        .Inst_in_MemWB(Inst_out_EXMem),
        .valid_in_MemWB(valid_out_EXMem),
        .PC4_in_MemWB(PC4_out_EXMem),
        .Rd_addr_MemWB(Rd_addr_out_EXMem),
        .ALU_in_MemWB(Addr_out),
        .DMem_data_MemWB(Data_in),
        .MemtoReg_in_MemWB(MemtoReg_out_EXMem),
        .RegWrite_in_MemWB(RegWrite_out_EXMem)
        .PC_out_MemWB(PC_out_MemWB),
        .Inst_out_MemWB(Inst_out_MemWB),
        .valid_out_MemWB(valid_out_MemWB),
        .PC4_out_MemWB(PC4_out_MemWB),
        .Rd_addr_out_MemWB(Rd_addr_out_MemWB),
        .ALU_out_MemWB(ALU_out_MemWB),
        .DMem_data_out_MemWB(DMem_data_out_MemWB),
        .MemtoReg_out_MemWB(MemtoReg_out_MemWB),
        .RegWrite_out_MemWB(RegWrite_out_MemWB)
    );

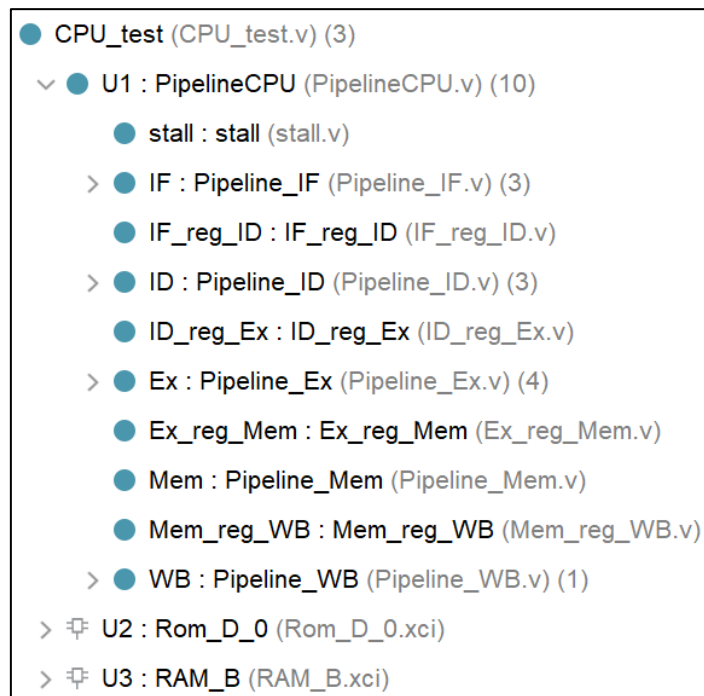
    Pipeline_WB WB(
        .PC4_in_WB(PC4_out_MemWB),
        .ALU_in_WB(ALU_out_MemWB),
        .DMem_data_WB(DMem_data_out_MemWB),
        .MemtoReg_in_WB(MemtoReg_out_MemWB),
        .Data_out_WB(Data_out_WB)
    );
endmodule

```

由于 Pipeline 的对外接口并未修改，因此 CSSTE.v 文件可以沿用 Lab5-3 的代码，不用修改（本报告就不计字数了）。

3.3 搭建仿真平台

参考 Lab5-03 的操作也能建立一个 CPU 仿真测试平台，对相关指令进行检测。利用 PipelineCPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台：



初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（h_mem.coe），查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。

因为流水线处理 Stall 的过程在 DEMO 物理测试的时候很难展示在报告中（连续多张图片很难讲清），因此分析的时候就用仿真图来说明，并随后贴出下板时的连续图来与仿真图对应。

（• 仿真图放在后面）。

3.4 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格（见实验数据分析）。

- ROM 中填充指令如下：

P_Mem (无冒险)

```

1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  00100093,
4  00100113,
5  00100193,
6  00100213,
7  00802283,
8  00108333,
9  0020C3B3,
10 40110433,
11 05C02483,
12 00327533,
13 00502223,
14 005325B3,
15 0AA3C613,
16 0012D6B3,
17 00147713,
18 0034E7B3,
19 00A50833,
20 0085C8B3,
21 00402903,
22 004629B3,
23 0016DA13,
24 00677AB3,
25 40128B33,
26 00150B93,
27 00986C33,
28 00B9CCB3,
29 0FFA7D13,
30 00390DB3,
31 002A5E33,
32 0AF9EE93,
33 001A0F33,
34 00802F83,
35 F81FF06F,
36 00000033,
37 00000033,
38 00000033;

```

h_Mem (有冒险)

```

1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  00100093,
4  00100113,
5  00100193,
6  00100213,
7  00802283,
8  00128333,
9  0020C3B3,
10 40708433,
11 05C02483,
12 00327533,
13 00502223,
14 005325B3,
15 0AA3C613,
16 00818663,
17 00000013,
18 00000033,
19 0012D6B3,
20 00147713,
21 0034E7B3,
22 00A50833,
23 0085C8B3,
24 00402903,
25 004629B3,
26 0016DA13,
27 00677AB3,
28 01071463,
29 00000013,
30 40128B33,
31 00150B93,
32 00986C33,
33 00B9CCB3,
34 0FFA7D13,
35 00390DB3,
36 002A5E33,
37 0AF9EE93,
38 001A0F33,
39 00802F83,
40 F6DFF06F,
41 00000033,
42 00000033,
43 00000033;

```

- RAM 中储存的数据 (D_mem.coe) 如下：

```

memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7DBFDB9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFFBCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7DBFDB9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300

```

部分测试 VGA 显示图片见实验结果分析

四、实验结果分析

此次物理验证类似 Lab5-3，主要看基本指令能否实现？寄存器数值是否根据对应指令正确修改（存在冒险）？以及流水线是否能正确处理 stall。

PC	Machine	Code Basic	Result
0x0	0x00100093	addi x1 x0 1	x1 = 0x1
0x4	0x00100113	addi x2 x0 1	x2 = 0x1
0x8	0x00100193	addi x3 x0 1	x3 = 0x1
0xc	0x00100213	addi x4 x0 1	x4 = 0x1
0x10	0x00802283	lw x5 8(x0)	x5 = 0x80000000
0x14	0x00128333	add x6 x5 x1	x6 = 0x80000001
0x18	0x0020C3B3	xor x7 x1 x2	x7 = 0x0
0x1c	0x40708433	sub x8 x1 x7	x8 = 0x1
0x20	0x05C02483	lw x9 92(x0)	x9 = 0xFFFFFFFF
0x24	0x00327533	and x10 x4 x3	x10 = 0x1
0x28	0x00502223	sw x5 4(x0)	mem[1] = 0x80000000
0x2c	0x005325B3	slt x11 x6 x5	x11 = 0x0
0x30	0x0AA3C613	xori x12 x7 170	x12 = 0xAA
0x34	0x00818663	beq x3 x8 12	go to 0x40
0x38	0x00000013	addi x0 x0 0	
0x3c	0x00000033	add x0 x0 x0	
0x40	0x0012D6B3	srl x13 x5 x1	x13 = 0x40000000
0x44	0x00147713	andi x14 x8 1	x14 = 0x1
0x48	0x0034E7B3	or x15 x9 x3	x15 = 0xFFFFFFFF
0x4c	0x00A50833	add x16 x10 x10	x16 = 0x2
0x50	0x0085C8B3	xor x17 x11 x8	x17 = 0x1
0x54	0x00402903	lw x18 4(x0)	x18 = 0x80000000
0x58	0x004629B3	slt x19 x12 x4	x19 = 0x0
0x5c	0x0016DA13	srl x20 x13 1	x20 = 0x20000000
0x60	0x00677AB3	and x21 x14 x6	x21 = 0x1
0x64	0x01071463	bne x14 x16 8	go to 0x6c
0x68	0x00000013	addi x0 x0 0	
0x6c	0x40128B33	sub x22 x5 x1	x22 = 0x7FFFFFFF
0x70	0x00150B93	addi x23 x10 1	x23 = 0x2
0x74	0x00986C33	or x24 x16 x9	x24 = 0xFFFFFFFF
0x78	0x00B9CCB3	xor x25 x19 x11	x25 = 0x0
0x7c	0x0FFA7D13	andi x26 x20 255	x26 = 0x0
0x80	0x00390DB3	add x27 x18 x3	x27 = 0x80000001
0x84	0x002A5E33	srl x28 x20 x2	x28 = 0x10000000
0x88	0x0AF9EE93	ori x29 x19 175	x29 = 0xAF
0x8c	0x001A0F33	add x30 x20 x1	x30 = 0x20000001
0x90	0x00802F83	lw x31 8(x0)	x31 = 0x80000000
0x94	0xF6DFF06F	jal x0 -148	go to 0x0
0x98	0x00000033	add x0 x0 x0	
0x9c	0x00000033	add x0 x0 x0	NOP -- 统一Pipeline各阶段
0xa0	0x00000033	add x0 x0 x0	

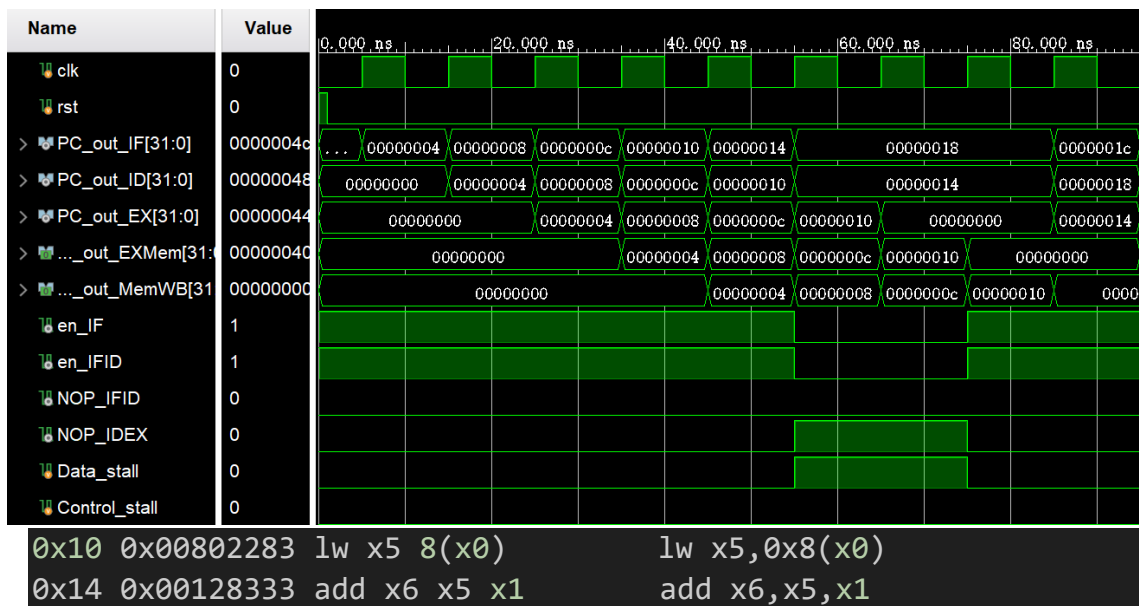
以上表格记录了 h_mem.coe 中的正常程序流（有冒险版本），并记录了运行后各寄存器的数值。

红色的表示此指令与前一指令发生 Data 冒险；

紫色的表示此指令发生 Control 冒险；

黄色的表示 没啥用。

(1) Data Hazard I :



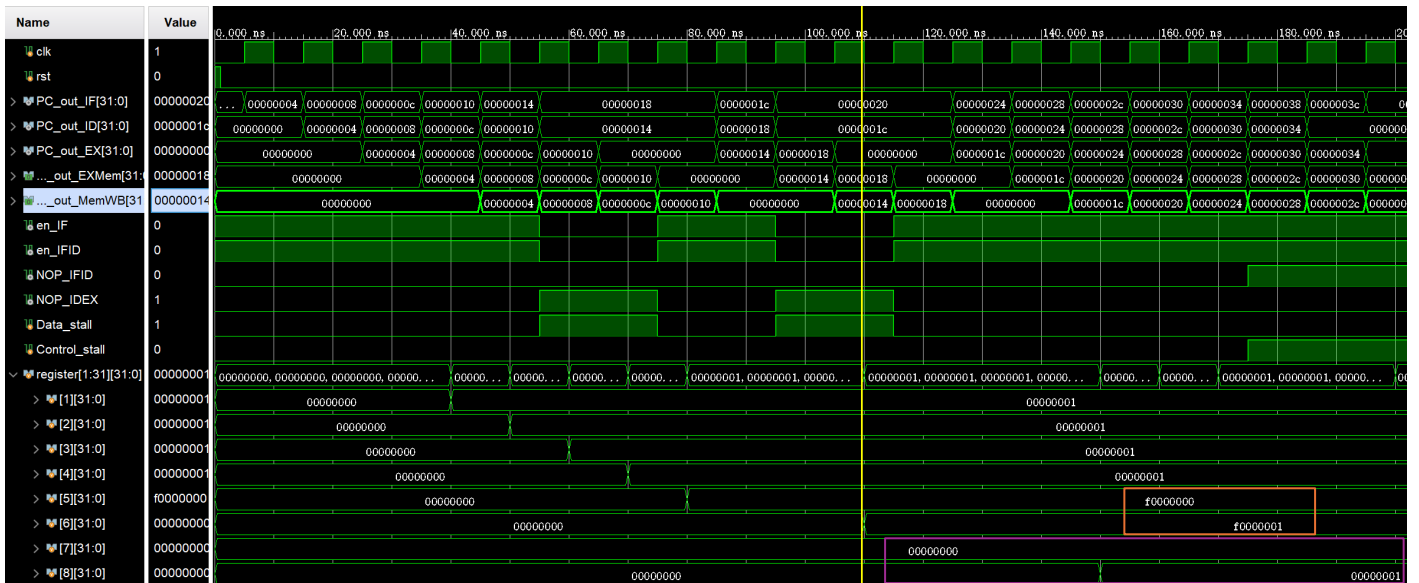
• 可以看到当 PC_out_ID 为 0x14 时（译码得到 Rs1used、Rs2used 等 stall 需要的信号）与 PC_out_EX=0x10 发生 Data Hazard ，此时是 Ex_Hazrad.

因此插入第 1 次 NOP，IF、ID 阶段停止读入指令/向下传输信号，注意下一 cycle 执行（Ex）阶段变成 NOP；

• 当 PC_out_EXMem = PC_out_Mem = 0x10 时，发生 Mem_Hazard，因此插入第 2 次 NOP，IF、ID 阶段停止读入指令/向下传输信号，下一 cycle 访存（Mem）阶段变成 NOP（此时 lw 数据已经成功读出）；

• 最后写回（WB）阶段 = 0x10，将 x5 的数值成功写回（**注意：ID 阶段对 Regs 的时钟信号进行取反，从而实现一个时钟周期前写后读的操作**），即 75ns 时 WB 写入 x5，80ns 时 ID 读出更新后的 x5，从而令下一 cycle 中 PC_out_EX = 0x14 时使用的 x5 是正确的。Now ， Data Hazard I has been solved.

```
Regs Regs_0(
    .clk(~clk_ID), // SIGNIFICANT!
    .rst(rst_ID),
    .Rs1_addr(Inst_in_ID[19:15]),
    .Rs2_addr(Inst_in_ID[24:20]),
    .Wt_addr(Rd_addr_ID),
    .Wt_data(Wt_data_ID),
    .RegWrite(RegWrite_in_ID),
    .Rs1_data(Rs1_out_ID),
    .Rs2_data(Rs2_out_ID),
    .all_regs_data(all_regs_data)
);
```



从后续的流水线可以看到（110ns），PC_out_MemWB=0x14 时写入了正确的 x6 数值 0xf0000001（不知道为啥仿真时 x5 load 出来是 0xf0000000，下板则是正确的 0x80000000），而非错误的 0x00000001（未处理冒险的结果）。

以下为 VGA 显示图片（👉👉顺序），后三个阶段只导出了 PC 信号，能够做到简单识别：



```

RV32I Pipelined CPU
===== If =====
pc: 0000001c inst: 40708433 valid: 0
===== Id =====
pc: 00000018 inst: 0020c3b3 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000014 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen:
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000020 inst: 05c02483 valid: 0
===== Id =====
pc: 0000001c inst: 40708433 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000018 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen:
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000014 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 80000001
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

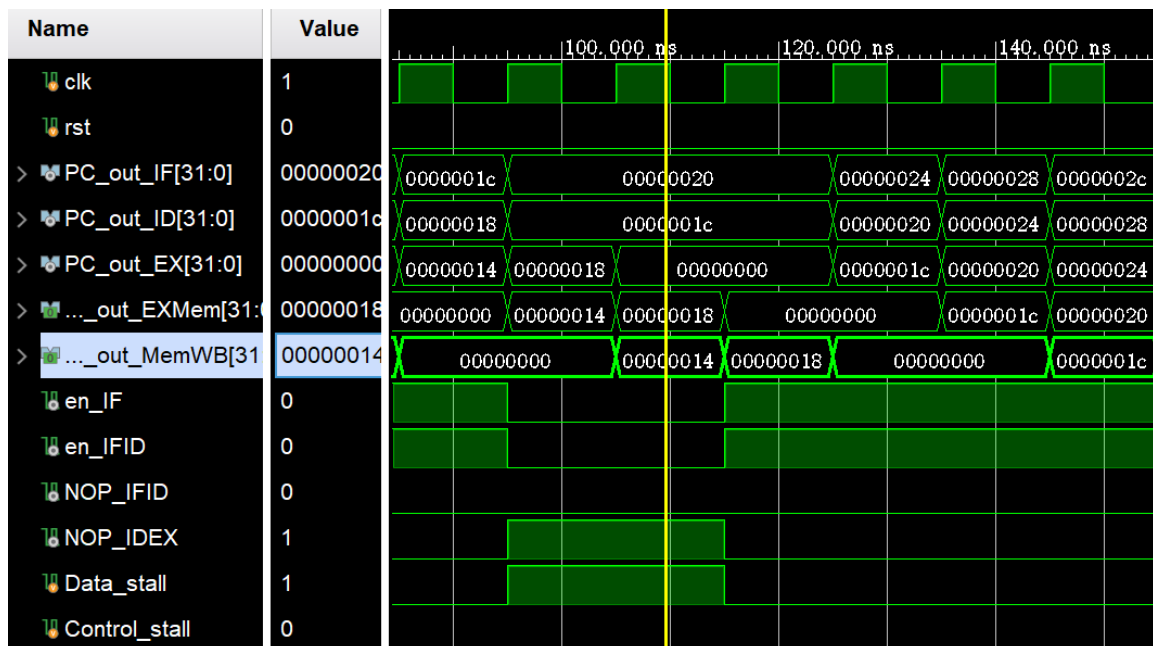
```

RV32I Pipelined CPU
===== If =====
pc: 00000020 inst: 05c02483 valid: 0
===== Id =====
pc: 0000001c inst: 40708433 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 80000001 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen:
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000018 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000014 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 80000001

```

以上 7 张图展示了处理 Data Hazard 的前因后果，最后 $x6 = t1 = 0x80000001$ 也是修成正果。

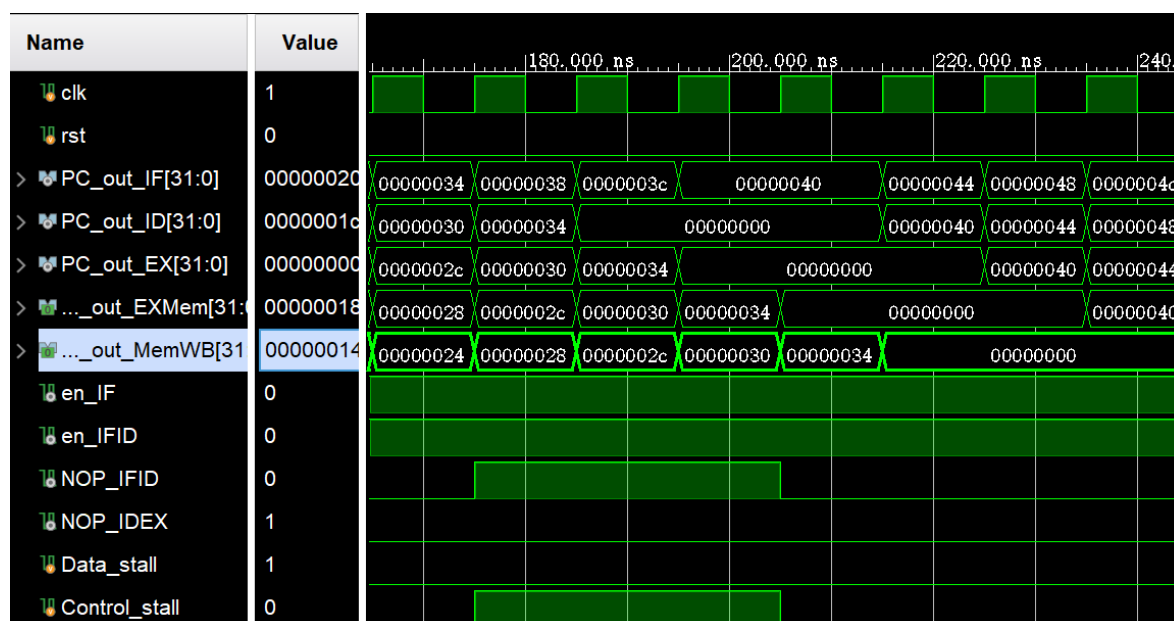
(2) Data Hazard II:



```
0x18 0x0020C3B3 xor x7 x1 x2      xor x7,x1,x2 #x7 = 0
0x1c 0x40708433 sub x8 x1 x7      sub x8,x1,x7 #x8 = 0x1
```

此处对 Data Hazard 的处理与上面一个基本一样，此处就不再详细说明了。
正确数据见 Data Hazard I 第二张图的紫色框框。

(3) Control Hazard I (beq):



```
0x34 0x00818663 beq x3 x8 12      beq x3,x8,loop1
0x38 0x00000013 addi x0 x0 0      addi x0,x0,0x0
0x3c 0x00000033 add x0 x0 x0      add x0,x0,x0
0x40 0x0012D6B3 srl x13 x5 x1      loop1:srl x13,x5,x1
```

首先，处理 Control Hazard 是在 IF 和 ID 之间插入 NOP，即取指（IF）是不会停止的。

可以看到当 PC_out_ID = 0x34 时，译码得到了 Branch_ID = 1（随后三个 cycle 插入 NOP）因此 NOP_IFID = 1，后续阶段不再读取新的指令。

PC_out_EXMem = 0x34 后得到了 PCSrc 来控制下一跳转地址，这可以解释 PC_out_IF 中 0x40 持续两个 cycle 的现象：第一次是正常顺序读取；第二次则是 PCSrc 控制 IF 取指，从而进行 beq 再次跳转到 0x40。

- Control Hazard 解决，可以看到除了 IF 的后续阶段并未执行 0x38、0x3c 两条指令，通过 NOP 直接跳转到 0x40;

```

RV32I Pipelined CPU
===== If =====
pc: 0000003b inst: 00000013
===== Id =====
pc: 00000034 inst: 00819663 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000030 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 0000002c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 0000002b inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000004

```

```

RV32I Pipelined CPU
===== If =====
pc: 0000003c inst: 00000033
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000034 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000030 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 000000aa
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 0000002c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000040 inst: 0012d6b3
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000034 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000030 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 000000aa

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000040 inst: 0012d6b3
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000034 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

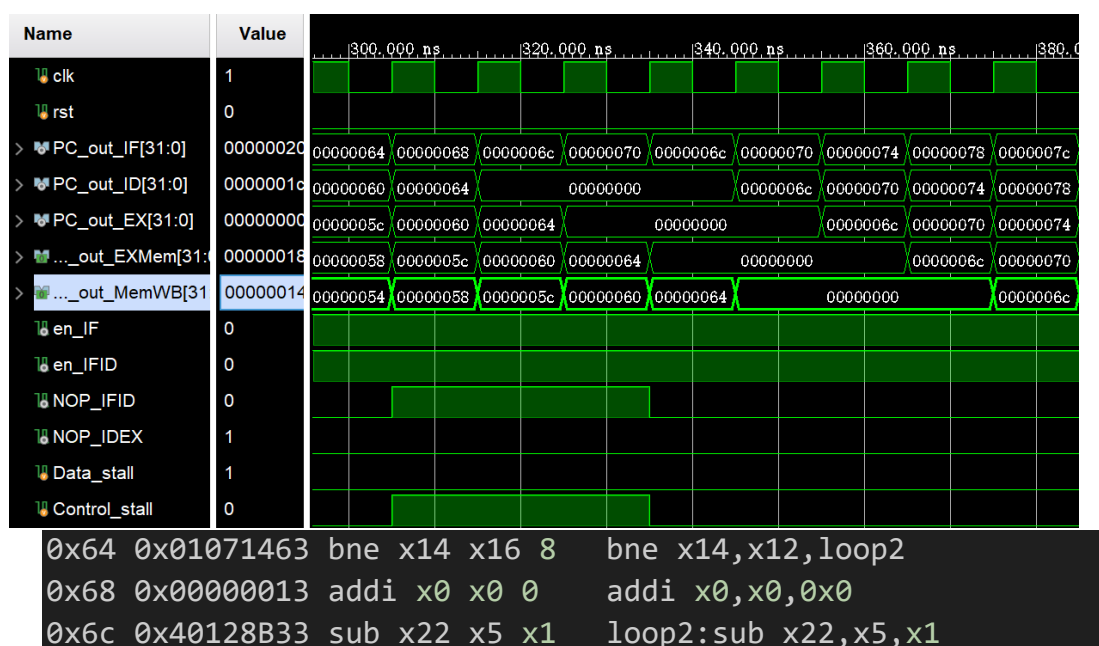
```

RV32I Pipelined CPU
===== If =====
pc: 00000044 inst: 00147713
===== Id =====
pc: 00000040 inst: 0012d6b3 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

以上 5 张图片展示了处理 beq 导致的 Control Hazard, 可以看到除了 IF 阶段, 后续 4 个阶段都实现了由 0x34 —— 0x40 的跳转, 没有对中间指令的冗余处理。

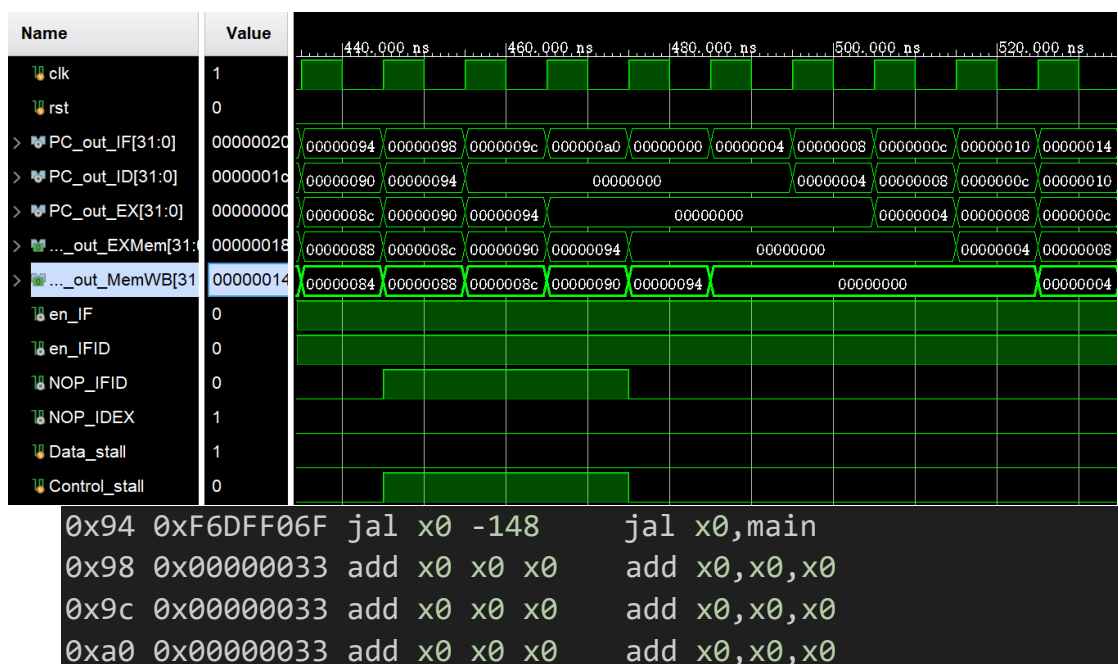
(4) Control Hazard II (bne):



此处处理与 beq 一样，但不同的是此时 bne 跳转指令只跳了 1 条，导致 IF 取到了超前的指令（0x70），怎么办？？？

没事，反正这没传到后续阶段，并且 PCSrc 随后会控制 IF 重新读取正确的下一条指令 0x6c，从此走上光明的道路。

(5) Control Hazard II (jal):



可以看到此处 jal 将后续 3 条指令都跳转了，且后续阶段也未将这 3 个无效指令读取，Control Hazard 也得到了处理。

```

RV32I Pipelined CPU
===== If =====
pc: 00000098 inst: 00000033
===== Id =====
pc: 00000094 inst: f6dfff0f valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000090 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 0000008c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 20000001
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000088 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 000000af

```

```

RV32I Pipelined CPU
===== If =====
pc: 0000009c inst: 00000033
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 00000000
===== Ex =====
pc: 00000094 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000090 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 00000008
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 0000008c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 20000001

```

```

RV32I Pipelined CPU
===== If =====
pc: 000000a0 inst: 00000033
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000094 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 40000000 alu_res: ffffffff
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000090 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000000 inst: 00100093
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000094 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000098

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000004 inst: 00100113
===== Id =====
pc: 00000000 inst: 00100093 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 80000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 80000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

以上 5 张图片展示了如何处理 jal 引起的 Control Hazard, 可以看到除了 IF 阶段, 后续 4 个阶段都实现了由 0x94 —— 0x00 的跳转。

最后, 这张照片也给出了所有寄存器的数据 (见实验结果分析的第一个表格), 其中 t1、s0 两处涉及 Data Hazard, 均成功处理!

实验成功~~~~

五、讨论与心得

最后一个实验报告也是写完了（不知道谁的 Lab6 报告先交了~~）。

说实话单周期、流水线 CPU 做到后面感觉像是复习？？特别是写报告的时候需要理清 stall 的处理原理还是比较困难的，但最后结合仿真还是搞得比较清楚了吧~~

该复习去了……

【并且我宣布 12.20，我自主完成了所有计组实验（真没开挂!）】